



PROGETTO DI SOFTWARE ENGINEERING FOR ARTIFICIAL INTELLIGENCE

**Analisi e Valutazione dell'Evoluzione Storica degli
Agent Context File tramite approccio basato su
Prompt Engineering e Classificazione Multi-LLM**

Professori

Prof. Fabio Palomba
Dott. Giammaria Giordano
Dott. Gianmario Voria

Autori

Chiara Puglia - 0522501984
Luigi Giacchetti - 0522502014
Luca Giuliano - 0522501931

Università degli Studi di Salerno
a.a 2025/2026

Contents

1	Stato dell'Arte	3
2	Analisi e Pre-Processing del Dataset	4
2.1	Evoluzione e Dinamiche dei Context File	7
3	Metodologia e Raccolta dei Dati	8
3.1	Estrazione delle diff dallo storico dei commit	8
4	Architettura di analisi delle diff	10
4.1	Fase di Acquisizione	11
4.2	Prompt Engineering	12
4.3	Chunking e Aggregazione	14
4.4	Comunicazione con modello	15
4.5	Parsing	16
4.6	Output	17
4.6.1	Analisi dell'output	18
4.6.2	Distribuzione dei punteggi per categoria	19
4.6.3	Primario rispetto al generale	20
4.6.4	Spostamento della distribuzione	21
4.6.5	Variazione della posizione in classifica	22
4.6.6	Matrice di co-occorrenza	23
4.6.7	Probabilità condizionata	24
4.6.8	Analisi dello status	25
4.6.9	Distribuzione dei maintenance type	26
4.6.10	Correction rispetto a Enhancement	28
4.6.11	Distribuzione dei punteggi per maintenance type	29
4.6.12	Traiettoria di manutenzione di un singolo ACF	30
5	Valutazione delle ambiguità	30
5.1	Metriche di accordo Cross-Model	31
5.2	Quantificazione dell'Ambiguità	32
5.3	Ambiguità dell'Asse di Manutenzione	34
5.4	Confronto con il Modello di Riferimento	35
5.5	Accordo aggregato	37
6	Golden Standard	38
6.1	Confronto tra risultati ottenuti tramite gli LLM con Golden Standard per Categoria	39
6.2	Confronto tra risultati ottenuti tramite LLM con Golden Standard per Main- tenance	40
6.3	Analisi delle Ambiguità vs Errore	42
7	Conclusioni	43
8	Sviluppi futuri	44

1 Stato dell'Arte

L'integrazione di agenti di codifica basati su LLM nello sviluppo Software, ha portato ad una rapida adozione dei file di contesto a livello di repository, come ad esempio CLAUDE.md e AGENTS.md che forniscono linee guida su architettura, comandi di build e convenzioni del codice. Tuttavia, l'adozione di questi artefatti risulta ancora in una fase iniziale e non esiste ancora tutt'oggi una struttura di contenuto standardizzata, a partire da una variazione negli stili di scrittura fino al condizionale. L'impatto reale di questi file sulle prestazioni degli agenti è ancora oggetto di indagine nella letteratura.

Da un lato, alcuni studi dimostrano che i file di contesto generati automaticamente dagli LLM tendono a ridurre il tasso di successo della risoluzione dei task e ad aumentare i costi di inferenza di oltre il 20%, mentre i file redatti dagli sviluppatori offrono miglioramenti prestazionali marginali, nonostante si spingano gli agenti a esplorare e testare il codice in maniera approfondita. Dall'altro lato, indagini incentrate sull'efficienza operativa evidenziano che, la presenza di file AGENTS.md comporta l'ottimizzazione dell'esecuzione, riducendo in questo modo il tempo di completamento delle attività del 28,64% e diminuendo il consumo di token in output del 16,58%.

Nonostante vari dibattiti sull'efficacia assoluta, la ricerca dimostra che questi file sono configurazioni dinamiche e mantenute attivamente, i cui commit e aggiornamenti servono a raffinare e correggere le istruzioni fornite alle intelligenze artificiali. Inserendosi in questo contesto, il seguente progetto mira al superamento dell'attuale approccio empirico, analizzando sistematicamente l'evoluzione di questi contesti operativi e misurando l'oggettività e la stabilità semantica delle direttive in essi contenute.

Il progetto si struttura in tre fasi metodologiche principali. La prima fase consiste nel tracciare l'evoluzione dei file di contesto nel tempo, estraendo in maniera automatizzata le differenze testuali (**diff**) dallo storico dei commit. Questo approccio consentirà di ignorare il testo inalterato e isolare esclusivamente le singole regole e istruzioni che gli sviluppatori hanno materialmente aggiunto, rimosso, modificato.

Nella fase successiva invece, le diff appena raccolte verranno processate da un LLM primario configurato con prompt strutturati. Il prompt avrà il compito di estrarre le **frasi chiave** e classificare la natura della modifica secondo specifiche euristiche predefinite.

Per quantificare il grado di non-determinismo proprio del linguaggio naturale, le frasi chiave estratte verranno fornite a 3 LLM differenti per verificare se i modelli classificano e interpretano le istruzioni allo stesso modo. Infine, le predizioni dei modelli saranno confrontate con un campione stratificato etichettato manualmente, misurando Precision, Recall ed F1-Score per determinare le divergenze e le ambiguità semantiche all'interno degli Agent Context File.

2 Analisi e Pre-Processing del Dataset

Il dataset di riferimento [1] è strutturato nel seguente modo:

- **content:** Il testo completo delle linee guida e delle istruzioni (come i file CLAUDE.md).
- **agent:** L'agente AI a cui il file è destinato (ad esempio, claude).
- **repository_owner:** L'utente o l'organizzazione proprietaria del repository su GitHub (es. Iamshankhadeep, tokoroten).
- **repository_name:** Il nome del progetto (es. ccseva, NovelDrive).
- **repo_url:** L'URL diretto alla pagina principale del repository.
- **branch:** Il ramo del progetto da cui è stato prelevato il file.
- **file_path:** Il percorso completo all'interno del progetto (spesso la root).
- **filename:** Il nome del file estratto (quasi tutti sono denominati CLAUDE.md).
- **file_url:** Il link diretto al file specifico su GitHub.
- **stars:** Il numero di "stelle" ricevute su GitHub (misura quanto il progetto è apprezzato).
- **forks:** Il numero di "fork" (quante volte il progetto è stato clonato/modificato da altri utenti).
- **created_at:** La data e l'ora di creazione del repository originale.
- **pushed_at / updated_at:** Date dell'ultimo aggiornamento o push di codice.
- **first_commit_date:** Data del primo commit nel progetto.
- **commit_count:** Il numero totale di commit eseguiti nel progetto, che aiuta a capirne la dimensione o l'attività.
- **content_commit_sha:** L'identificativo esatto (hash) del commit a cui corrisponde la versione del file scaricata.

Per garantire la coerenza e la qualità dei dati impiegati per le successive analisi, il dataset originale (composto inizialmente da 2303 istanze) è stato sottoposto ad una pipeline di pre-processing.

In primo luogo, sono state analizzate le colonne **content** (il corpo del file di contesto) e **content_commit_sha** (necessario per il tracciamento univoco della versione testuale nel tempo). Le righe che presentavano valori nulli (NaN) o stringhe vuote in questi campi sono state rimosse, in quanto un'osservazione priva di istruzioni risulta inusabile per l'estrazione delle regole semantiche.

Successivamente, si è tenuto conto del fatto che il solo popolamento della colonna **content** non garantisce l'effettiva utilità del file (es. file creati come semplici *placeholder*). È stata quindi calcolata la lunghezza in parole di ciascun testo ed è stata imposta una soglia

minima: tutte le righe con un `content` inferiore a 10 parole sono state scartate. A valle di queste prime fasi di pulizia testuale, il dataset è stato ridotto a 2179 osservazioni. Infine, è stata condotta una rapida verifica sull'integrità dei campi `file_path` e `filename` per confermare la corretta estrazione strutturale dei dati.

Per assicurarsi che le istruzioni analizzate derivassero da progetti con una reale storicità e non da repository sperimentali abbandonate subito dopo la creazione, si è studiata la distribuzione dell'attività di sviluppo tramite la variabile `commit_count` visualizzabile dal grafico seguente:

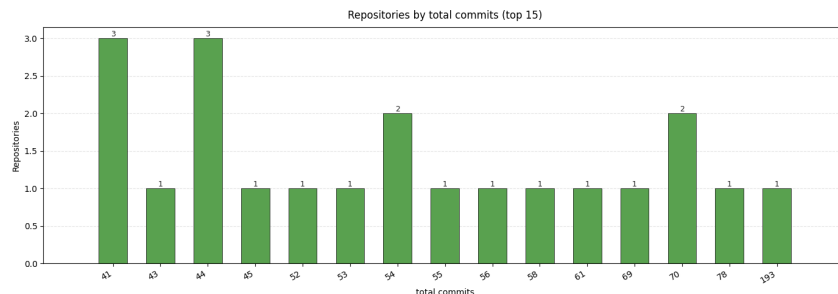


Figure 1: Distribuzione delle repositories del dataset

Come prima indagine esplorativa, è stato generato il barplot seguente che riguarda le 15 repository che ancora tutt'ora risultano attive (Figura 2). Il grafico ha evidenziato una forte disomogeneità nei commit, a partire dalle repository di punta fino ad arrivare al resto del dataset.

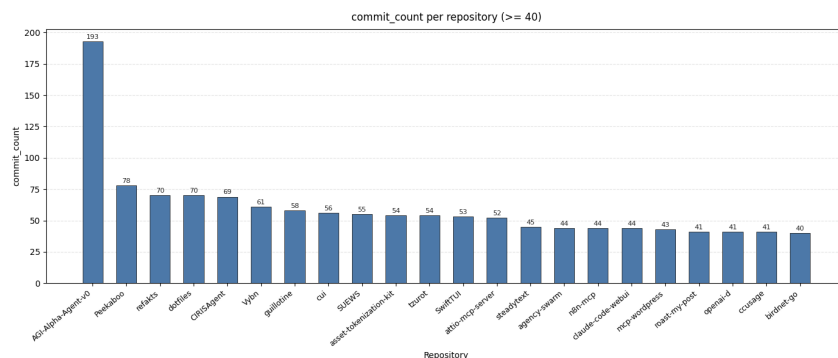


Figure 2: Grafico delle repository che risultano ancora attive

Per prendere una decisione sulla soglia di filtraggio da applicare, l'analisi si è concentrata sulla distribuzione globale. L'istogramma e la Funzione di Ripartizione Empirica (CDF) riportati in Figura 3 mostrano l'andamento del numero di repository al variare dei commit totali.

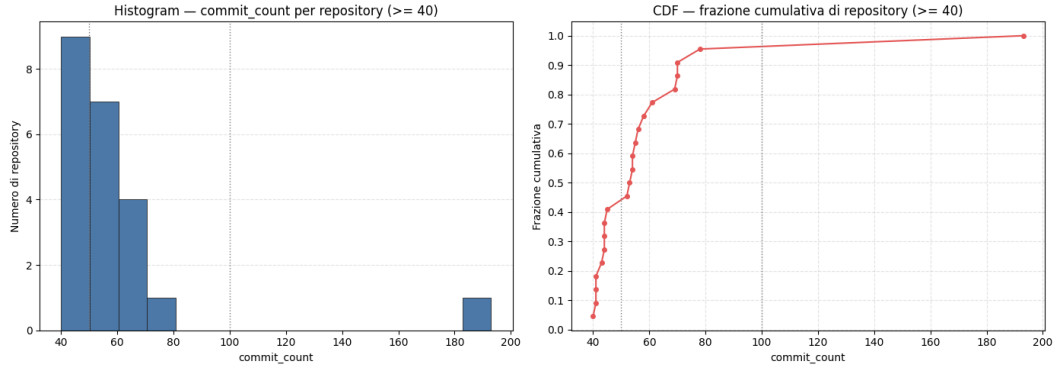


Figure 3: Istogramma e CDF della variabile `commit_count` per le repository che indica le potenziali soglie di filtraggio analizzate.

Sulla base delle evidenze statistiche emerse da questi grafici, molte repository possedevano uno storico di attività esiguo per supportare un'analisi evolutiva significativa. Pertanto, si è scelto di imporre una **soglia di 50 commit** complessivi. In seguito sono state riportate nel dettaglio le repository che rispettano tale soglia. Tutte le repository che non raggiungevano tale numero, sono state rimosse dal dataset.

Soglia: 50

Repository che superano la soglia: 13/1897

MontrealAI/AGI-Alpha-Agent-v0	193
steipete/Peekaboo	78
devill/refakts	70
baleen37/dotfiles	70
CIRISAI/CIRISAgent	69
zoedolan/Vybn	61
evmts/guillotine	58
BMPixel/cui	56
UMEP-dev/SUEWS	55
settlemint/asset-tokenization-kit	54
lbds137/tzurot	54
bannzai/SwiftTUI	53
kesslerio/attio-mcp-server	52

Righe nel dataset prima del filtro: 2179

Righe nel dataset dopo il filtro: 14

Al termine di questo di filtraggio, i dati rimanenti sono stati considerati validi e salvati in un nuovo file strutturato (`dataset_cleaned.csv`), pronto per essere elaborato dalla pipeline Multi-LLM nelle fasi successive del lavoro.

Successivamente alla creazione del `dataset_cleaned.csv`, l'attenzione si è spostata sullo studio dell'evoluzione temporale dei *context file* all'interno delle repository selezionate. A

tale scopo, è stato analizzato il file `git_history_diffs.json`, il quale raccoglieva lo storico delle modifiche per 12 repository, totalizzando 1073 commit complessivi.

Una prima ispezione di questi dati storici ha permesso di individuare e gestire lievi anomalie strutturali, come la presenza di commit duplicati. Nello specifico, è stata rilevata e normalizzata una singola occorrenza nella repository `CIRISAI/CIRISAgent`, che presentava un hash duplicato.

Al fine di garantire la massima coerenza nelle successive analisi testuali (in particolar modo per le metriche NLP e le elaborazioni tramite LLM), è stato introdotto un ulteriore livello di filtraggio basato sulla lingua dei messaggi di commit. Si è deciso di escludere le repository che presentavano storici con massiccia presenza di caratteri in lingua giapponese (hiragana/katakana) o coreana (hangul), poiché avrebbero introdotto "rumore" e compromesso i calcoli successivi. Questa operazione ha comportato la rimozione di due progetti:

- **bannzai/SwiftTUI**: escluso a causa di 47 commit scritti nelle lingue asiatiche citate.
- **baleen37/dotfiles**: escluso per la presenza di 36 commit analoghi.

Conclusa questa fase ulteriore di filtraggio, l'analisi dello storico dei commit si è concentrata esclusivamente sulle 10 repository risultate idonee.

2.1 Evoluzione e Dinamiche dei Context File

Per comprendere concretamente come le istruzioni fornite agli agenti AI vengano gestite e mantenute nel tempo, si è studiata la distribuzione globale degli *status* operativi registrati per i context file durante i vari commit. I risultati hanno evidenziato la seguente ripartizione:

- **Modificati (modified)**: 914 occorrenze
- **Rimossi (removed)**: 5 occorrenze
- **Aggiunti (added)**: 2 occorrenze
- **Rinominati (renamed)**: 1 occorrenza

La netta prevalenza dell'operazione di modifica (914 occorrenze contro le pochissime operazioni strutturali) suggerisce in modo evidente che file come `CLAUDE.md` non sono concepiti come artefatti statici inseriti all'inizio del progetto. Al contrario, si tratta di documenti "vivi" che gli sviluppatori aggiornano e raffinano continuamente in risposta all'evoluzione del codice sorgente.

Un'analisi più granulare sulle singole repository ha permesso di isolare i commit con il maggior impatto quantitativo, confermando questa ipotesi. A titolo esemplificativo, sono emersi due casi di particolare interesse in cui il context file ha subito revisioni massicce:

- Nella repository **BMPixel/cui**, l'autore *bmpixel* ha effettuato un commit chiave (associato al messaggio: "*Add multiple CLAUDE.md*") che ha generato ben 617 cambiamenti sul context file, distribuiti in 38 righe aggiunte e 579 rimosse.

- Nel caso di **CIRISAI/CIRISAgent**, l'autore *Eric* ha effettuato un aggiornamento critico in concomitanza con un importante refactoring del codice (messaggio: *"Major refactor: Eliminate Dict[str, Any] and achieve pytest green (#146)"*). Questo singolo evento ha comportato ben 859 variazioni all'interno delle istruzioni per l'AI, suddivise in 520 aggiunte e 339 rimozioni.

3 Metodologia e Raccolta dei Dati

3.1 Estrazione delle diff dallo storico dei commit

La fase di estrazione delle differenze testuali (**diff**) è stata automatizzata mediante lo sviluppo di uno script Python dedicato (`github_history.py`), progettato per interfacciarsi con le API REST di GitHub. L'obiettivo principale di questa procedura è isolare e tracciare le specifiche modifiche semantiche apportate ai file di contesto (prevalentemente `CLAUDE.md` e `AGENTS.md`) nel corso del tempo, partendo dal nuovo dataset strutturato (`dataset_cleaned.csv`).

Il processo inizia con l'individuazione esatta del file all'interno di ogni repository tramite le istanze presenti nel dataset: proprietario, nome della repository, percorso del file e l'hash del commit di riferimento (`content_commit_sha`). Per assicurare un'elevata percentuale di successo nel recupero dei dati, lo script implementa una logica di tolleranza agli errori. Qualora il branch di riferimento non sia disponibile o restituisca un errore HTTP 404, la funzione `collect_window_with_fallback` interroga l'API per identificare il branch di default del progetto e ritenta l'estrazione.

Una volta acquisita la cronologia del file, lo script seleziona una specifica "finestra" di commit (parametrizzabile, ad esempio gli ultimi 3, 5 o 10 commit precedenti a quello di riferimento). Per ogni coppia di commit consecutivi (`base_sha` e `head_sha`), viene effettuata una chiamata all'endpoint di comparazione di GitHub per estrarre le modifiche effettive.

Un aspetto critico di questa fase riguarda la natura dei dati restituiti da GitHub. Le modifiche vengono fornite sottoforma di "unified diff" (patch), che includono non solo il testo alterato, ma anche metadati strutturali come gli header dei file (`+++`, `---`) e gli indicatori di blocco (`@@`). Poiché l'obiettivo successivo del progetto è far analizzare questi testi a un LLM, fornire rumore sintattico comprometterebbe la precisione della classificazione semantica.

Per ovviare a questo problema, è stata sviluppata una logica di parsing mirata all'interno della funzione `parse_patch_lines`:

```
def parse_patch_lines(patch: Optional[str]) -> Tuple[List[str], List[str]]:
    # Parse a unified diff patch into added and removed instruction lines.

    # Skips hunk headers (@@) and file headers (+++/--). Returns stripped text
    # so callers get clean sentences ready for LLM processing.\
```



```

if not patch:
    return [], []
added: List[str] = []
removed: List[str] = []
for raw_line in patch.splitlines():
    if raw_line.startswith("+++") or raw_line.startswith("---"):
        continue
    if raw_line.startswith("+"):
        text = raw_line[1:].strip()
        if text:
            added.append(text)
    elif raw_line.startswith("-"):
        text = raw_line[1:].strip()
        if text:
            removed.append(text)
return added, removed

```

Come si evince dal codice, l'algoritmo itera su ciascuna riga della patch, ignorando i metadati e raccogliendo in due liste separate esclusivamente le istruzioni aggiunte (prefisso +) e quelle rimosse (prefisso -). L'operazione di `strip()` assicura che il testo risultante sia una frase pulita e pronta per l'elaborazione del modello del linguaggio.

Quest'operazione di estrazione delle diff utilizzando le API di GitHub, ha reso necessaria l'implementazione di una rigorosa gestione dei limiti di traffico (Rate Limiting). La funzione `rest_get_json` valuta gli header di risposta HTTP (Retry-After e X-RateLimit-Reset) e, in caso di errori 403 o 429, sospende l'esecuzione dello script calcolando dinamicamente i secondi di attesa necessari per il ripristino del token. Questo garantisce che l'estrazione non si interrompe prematuramente e non corrompe il dataset in costruzione.

In conclusione, l'intero output della pipeline — che comprende metadati dell'autore, messaggi di commit, URL di comparazione e le righe testuali pulite (aggiunte e rimosse) — viene serializzato riga per riga in un file JSONL (`git_history_diffs.jsonl`). Questa scelta architetturale permette di gestire in modo efficiente l'elevato volume di dati e fornisce un formato strutturato ottimale per alimentare i prompt Multi-LLM previsti nella fase successiva della metodologia.

Successivamente alla fase di estrazione, i dati grezzi raccolti riga per riga nel file JSONL vengono processati da un ulteriore script Python (`transform_diffs.py`). L'obiettivo di questo passaggio è convertire l'elenco sequenziale e frammentato in un formato JSON standard gerarchico e strutturato (`git_history_diffs.json`) che permette di ricostruire e preservare le relazioni gerarchiche intrinseche tra le repository, la loro evoluzione storica (i commit) e i singoli file testuali modificati. Fornire una struttura aggregata e coesa è cruciale per la fase successiva: quando i prompt Multi-LLM dovranno valutare le divergenze e le ambiguità semantiche, essi avranno accesso non solo a una modifica isolata, ma all'intera evoluzione direttiva della repository, garantendo valutazioni nettamente più accurate e contestualizzate.

A livello operativo, lo script esegue la lettura del file JSONL e riorganizza le singole entrate raggruppandole prima per repository (combinando il proprietario della repository e il nome

del progetto) e successivamente per "commit", identificato univocamente dalla coppia di hash base e head. Durante questa fase di parsing, l'algoritmo ordina cronologicamente **le diff** tramite un indice dedicato e aggrega in modo accurato tutti i dettagli dei file modificati (conteggio di aggiunte e rimozioni, nomi precedenti dei file in caso di rinominazioni, e URL diretti per l'accesso ai file grezzi su GitHub). Il risultato di questa elaborazione è un output in cui, per ogni repository, vengono mappati dinamicamente tutti i commit rilevanti, i metadati aggiornati e i pattern dei file di contesto individuati.

4 Architettura di analisi delle diff

Gli **Agent Context File** (ACF) sono i documenti attraverso cui un progetto software fornisce istruzioni e contesto operativo agli agenti di codifica; assumono nomi convenzionali quali `CLAUDE.md`, `AGENTS.md` o `copilot-istruzioni.md`. Questi file sono soggetti a un processo di revisione continua: ciascuna modifica (**diff**) ne documenta un'evoluzione puntuale, come un comando di compilazione, la precisazione di un vincolo architetturale, l'aggiunta di direttive di testing. L'identificazione del contesto semantico di tali modifiche costruisce il presupposto metodologico per un'analisi sistematica delle dinamiche di crescita e mutamento degli ACF.

Con il termine **label**, o anche etichetta, si disegna la categoria semantica che sintetizza l'aspetto dell'ACF interessato da una determinata diff. Il modulo di analisi ha il compito di assegnare automaticamente a ciascuna modifica presente una o più etichette, selezionata da una **tassonomia di sedici categorie** (Build and Run, Testing, Architetture, ecc). L'estrazione delle label rappresenta l'operazione centrale dell'intera architettura, in quanto converte un flusso non strutturato di modifiche testuali in una rappresentazione strutturata e interrogabile sull'evoluzione dell'ACF.

Sul piano architetturale, il compito di classificazione è delegato a un LLM (*Large Language Model*) pre-addestrato guidato da un prompt strutturato e servito tramite un API di Ollama. La motivazione alla base di questa scelta sono:

1. **Assenza di Dataset:** Un modello per un addestramento supervisionato ha necessità di una buona mole di dati;
2. **Flessibilità:** Rende più facile le modifiche in fase di configurazione senza preoccuparsi di dover ri-addestrare un modello;
3. **Portabilità:** l'utilizzo di Ollama consente di eseguire sia in locale che in cloud, con modelli intercambiabili.

L'estrazione delle label è realizzata in una pipeline, strutturata nel seguente modo:

- **Acquisizione:** acquisisce i diff dai formati eterogenei in cui si presentano e li riconduce a una forma unitaria, assegnando a ciascuno un identificativo;
- **Prompt Engineering:** definisce le istruzioni fornite al modello e le specifiche delle sedici categorie, con lo scopo di comunicare il compito di classificazione in modo non ambiguo;

- **Chunking:** segmenta i diff la cui lunghezza eccede la finestra di contesto del modello e ricompone in un unico vettore i punteggi prodotti dai singoli segmenti;
- **Comunicazione con il modello:** gestisce l'interazione con LLM, regolando la frequenza delle richieste e governando errori transitori e retry, così da garantire affidabilità dl dialogo su rete;
- **Parsing:** interpreta l'output del modello e applica strategie di recupero dei dati, al fine di preservare le classificazioni valide;
- **Output:** Salva i risultati sul disco finisco con modalità che permettono la ripresa di un'esecuzione senza dover ripetere l'elaborazione.

4.1 Fase di Acquisizione

Prima di descrivere il trattamento dei dati è opportuno precisare la natura del compito che l'architettura intende risolvere. Un singolo diff raramente interessa una sola categoria: una modifica può toccare simultaneamente più aspetti dell'ACF, come un comando di compilazione e, contestualmente, la sua documentazione. Il compito è quindi formulato come un **classificatore multi-label con punteggio continuo**: a ognuna delle sedici categorie della tassonomia[1] si assegna un punteggio sull'intervallo $[0.0, 1.0]$ che ne esprime la pertinenza.

I punteggi non formano una distribuzione di probabilità, ma ogni score assegnato a ogni singola categoria hanno valori indipendenti l'uno dall'altro. Da questi *score* derivano due nuove grandezze: la **primary_category**, ovvero la categoria con lo score massimo e le **flagged_categories** che sono le categorie il cui score supera la soglia **per-category-threshold**, di default fissata a 0.50. Il modello si limita a produrre lo score per ogni categoria; la decisione se considerarla come *flagging* spetta poi al confronto con la soglia. Tenere separati i due passaggi permette di rendere il classificatore più o meno conservativo spostando semplicemente la soglia, senza modificare il prompt o il modello.

La formulazione appena descritta permette di caratterizzare l'oggetto della modifica, ma non la sua motivazione. Le due dimensioni sono distinte e indipendenti: una modifica ai comandi di build può essere la riparazione di un comando non più funzionante, l'adeguamento a un nuovo toolchain o la riformulazione in una forma più leggibile, restano in tutti e tre i casi una modifica della categoria **Build and Run**. Il topic non determina la motivazione, e assumere il contrario significa sovrapporre due dimensioni distinte dall'evoluzione degli ACF, precludendo l'analisi delle dinamiche di manutenzione. Dato che la motivazione a qualificare la natura evolutiva dell'artefatto, dato che un file è sottoposto a ripetute riparazioni, descrive una dinamica di progetto diversa da uno progressivamente arricchito, anche quando entrambi insistono sulle medesime categorie tematiche. Il compito è stato esteso su due fronti, valutati dal modello in un'unica risposta: l'asse delle **categorie**, che esprime il topic della modifica con semantica multi-label, e l'asse dei **maintenance type**, che ne esprime la motivazione secondo la tassonomia normativa.

I due assi differiscono non solo nel significato ma anche nella struttura statistica attesa, e il prompt codifica tale differenza in modo esplicito. L'asse delle categorie è propriamente

multi-label: la presenza simultanea di più categorie con punteggio elevato è l'esito atteso e non un'anomalia. L'asse della manutenzione è invece quasi mutualmente esclusivo, poiché a una modifica corrisponde di norma una sola motivazione dominante; ci si attende dunque che esattamente un tipo si collochi nella fascia alta (≥ 0.60) e che i restanti rimangano nella fascia bassa. Entrambe le aspettative sono comunicate al modello come vincoli espliciti, così da evitare che la logica multi-label venga trasferita per analogia sull'asse a cui non si applica.

Le diff possono arrivare in formati eterogenei, prodotte da estrazioni diverse: a volte raggruppati per commit, a volte in un elenco unico, a volte con un record per riga; inoltre lo stesso dato, per esempio il testo di *patch* o l'hash del commit, può comparire sotto nomi di campi differenti. La fase di acquisizione ha il compito di ricondurre questa varietà in una forma unica, su cui tutti gli stadi successivi possono lavorare senza preoccuparsi della provenienza.

Per riuscirci, la componente di acquisizione segue i principi di robustezza: è tollerante agli input ricevuti e rigoroso rispetto agli output prodotti. Riconosce automaticamente il formato del file, provando in cascata i possibili nomi di campo finché trova il dato cercato e, se incontra una riga corrotta o illegibile, la salta senza interrompere l'intera elaborazione. Da ogni diff vengono così estratti pochi campi essenziali: l'identificativo, hash e messaggio commit, nome del file testo della modifica e data del commit in formato ISO 8601. Quest'ultimo campo, superfluo in una formulazione a un solo asse, diventa necessario con l'introduzione dell'asse di manutenzione, che è intrinsecamente temporale: distinguere una riparazione reattiva da un adeguamento proattivo presuppone infatti di sapere quando la modifica sia avvenuta e quali interventi l'abbiano preceduta.

A ciascuna diff viene infine assegnato un identificativo stabile, costruito a partire dall'hash del commit e dal nome del file. Questo identificativo garantisce il "resume" della pipeline: se un'esecuzione viene interrotta e ripresa successivamente, le diff già elaborate vengono riconosciute dal loro identificativo e si prosegue al successivo, evitando di riprocessare lo stesso lavoro.

4.2 Prompt Engineering

La classificazione dei commit è affidata a un modello linguistico pre-addestrato: questa scelta è dettata dall'assenza di un **dataset** etichettato di dimensioni sufficienti, dall'onerosità di costruirne uno e dalla necessità di poter rivedere e raffinare le sedici categorie senza un nuovo ciclo di addestramento. In questo contesto, il **prompt engineering** diventa la leva principale per orientare il comportamento del modello: non disponendo di pesi da ottimizzare, è attraverso la formulazione del computo che si codificano le definizioni, vincoli e criteri di scoring. Il prompt si articola in un **system prompt** statico con compito e regole, e uno **user prompt** dinamico con diff e metadati. Le sedici categorie sono **esternalizzate** in un file di configurazione (categories.json): per ognuna, oltre al nome, è presente una descrizione strutturata con una clausola "Look for:" per i segnali positivi e clausole "NOT this if" che disambiguano le categorie. Le descrizioni complete, con esempi **few-shot**, sono iniettate

inline nel prompt così che il modello disponga dell'intero contesto semantico; la scelta è di manutenibilità, poiché i confini fra categorie si raffinano modificando dati e non il codice.

Il prompt contrasta due fallimenti tipici degli LLM nello scoring: il primo è la **compressione dei punteggi** verso la fascia centrale (0.3-0.5), un bias per cui il modello evita i valori estremi e tende alla media, rendendo la soglia poco discriminante. Per correggerlo, il prompt definisce una **scala di scoring esplicita** che associa a ogni livello una descrizione verbale e impone alla categoria più pertinente di collocarsi nella fascia alta (≥ 0.70). Il secondo è quello di **astensione**, ovvero la restituzione di un vettore interamente nullo su un diff non vuoto, vale a dire un falso negativo sistematico. Per contrastarla, il prompt introduce un vincolo di **anti-azzeramento**, una regola dedicata alle cancellazioni pure. Per tale regola viene effettuata una classificazione in base al *topic* del contenuto rimosso, e un insieme di ancore tematiche, i cosiddetti *topic anchor* che mappano i pattern ricorrenti sulla categoria attesa. Infine, viene fissata una clausola che vincola l'output in un unico formato JSON, senza testo libero né delimitatori Markdown.

Allo stesso criterio di esternalizzazione risponde il secondo asse, quello della motivazione di manutenzione, le cui definizioni sono iniettate dal file `modification_request.json`. Esso adotta la tassonomia della *Modification Request* dello standard ISO/IEC/IEEE 14764:2022 [2]. Ancorare l'asse a uno standard, invece di costruire una tassonomia *ad hoc*, dà alle definizioni una *validità di costrutto* che il giudizio soggettivo degli autori non avrebbe, e rende i risultati confrontabili con la letteratura di *software maintenance*. Lo standard articola la richiesta di modifica su due livelli: al primo essa è classificata come **Correction** o **Enhancement**, al secondo le è assegnato un *maintenance type*. La distinzione fra le due classi non riguarda la dimensione dell'intervento, ma il suo rapporto con i requisiti: una Correction ripara il software rispetto a requisiti, un Enhancement implementa un requisito nuovo.

L'applicazione diretta della tassonomia incontra però una difficoltà, la cui risoluzione è una scelta progettuale non banale: il tipo **adaptive** è **dual-parented**, poiché discende sia da Correction sia da Enhancement a seconda che l'adeguamento sia reattivo rispetto a un ambiente *già* mutato oppure proattivo rispetto a uno *in via di* mutamento. L'ambiguità è irrilevante per un lettore umano ma incompatibile con un classificatore: restituendo **adaptive**, la classe genitore andrebbe inferita a posteriori, reintroducendo per via euristica la soggettività che l'adozione dello standard mirava a eliminare. Il tipo ambiguo è stato perciò **scisso in due target distinti**, **adaptive (correction)** e **adaptive (enhancement)**, disambiguati nel prompt da un criterio temporale. La classe genitore diventa così **deterministicamente derivabile** dalla foglia (Tabella 1), mentre ai fini del *reporting* le due foglie sono ricomposte in un unico **base-type adaptive**, affinché la granularità introdotta per disambiguare non frammenti le statistiche finali.

Foglia (target del modello)	Classe genitore	Base type
corrective	Correction	corrective
preventive	Correction	preventive
adaptive (correction)	Correction	adaptive
adaptive (enhancement)	Enhancement	adaptive
additive	Enhancement	additive
perfective	Enhancement	perfective

Table 1: Le sei foglie della tassonomia della richiesta di modifica, ridotte a cinque *base type* in fase di reporting. Lo sdoppiamento di **adaptive** rende la seconda colonna una funzione della prima.

I due vettori sono prodotti in un'unica risposta: la scelta dimezza il costo di inferenza e garantisce che entrambi i giudizi derivino dalla medesima lettura della diff. Il vincolo cruciale è quello di **indipendenza fra gli assi**, poiché il topic non detta la motivazione, che va decisa da ciò che la modifica *fa* al software assumendo il `commit_message` come segnale forte. Il confine più insidioso, quello fra **corrective** e **perfective**, è risolto leggendo lo stato precedente già contenuto nella diff: se il contenuto rimosso era *errato* e la modifica lo ripara si ha **corrective**, se era *valido* e la modifica lo rende solo più chiaro si ha **perfective**.

Tale criterio resta però limitato all'orizzonte della singola diff, che non mostra la storia del file: una modifica che revoca una regola introdotta tre commit prima è, letta isolatamente, indistinguibile da un riordino stilistico. Lo *user prompt* veicola perciò un campo opzionale `prior_changes_to_this_file` con le tre modifiche precedenti al medesimo file, ridotte a data e messaggio di commit per contenere il costo in token: l'informazione richiesta è la *traiettoria*, non il contenuto pregresso. Una modifica che inverte o ritratta una regola precedente è **corrective**, non **perfective**. Poiché il contesto storico rischierebbe di contaminare l'altro asse, il prompt fissa una regola di **isolamento**: la traiettoria vale per la sola motivazione e non deve mai innalzare il punteggio di una categoria, che va valutata strettamente sulle righe modificate della diff corrente.

La clausola testuale che impone un unico oggetto JSON esprime infine un'intenzione, non una garanzia. Ad essa si affianca perciò uno **schema JSON**, costruito dinamicamente sulle etichette in uso e trasmesso al servizio come parametro `format`, che vincola la decodifica alla grammatica dichiarata. Poiché lo schema fissa i nomi delle etichette mediante clausole **enum** generate dalle stesse liste che alimentano il prompt, l'invenzione di un'etichetta inesistente diviene strutturalmente impossibile anziché soltanto vietata a parole.

4.3 Chunking e Aggregazione

Per evitare troncamenti indesiderati in output, il sistema adotta un **adaptive context budgeting** [3]: all'avvio interroga il modello per leggerne la finestra reale, stima i token del prompt e ne deriva un limite di sicurezza per i chunk, riducendolo automaticamente se l'utente ne richiede uno maggiore. Misurare il contesto invece di assumerlo evita la classe di errori più insidiosa ovvero i JSON troncati o le categorie mancanti. Tale passaggio è

rappresentato dalla seguente funzione:

Listing 1: Taglio boundary-aware con overlap

```
if current_len + line_len > max_chars and current_len > 0:
    cut_at = current_start + current_len
    if (last_boundary_end is not None
        and last_boundary_end > current_start
        and (last_boundary_end - current_start) >= max_chars / 2):
        cut_at = last_boundary_end
    chunks.append(close_chunk(cut_at))
    overlap_start = max(0, cut_at - overlap_chars)
    current_start = overlap_start
    current_len = cut_at - overlap_start
    last_boundary_end = None
```

Ogni chunk viene classificato indipendente, producendo un vettore di score: questi vettori vanno poi ricomposti in un unico vettore che descrive l'intera diff. La ricomposizione avviene categoria per categoria: per ciascuna delle sedici categorie si combinano i punteggi che ha ottenuto nei vari chunk. Il criterio con cui combinarle è una strategia di aggregazione, e ne sono previste tre, selezionabili a runtime secondo lo **strategy pattern**.

La strategia di default è la **max**, che assegna a ogni categoria il punteggio più alto fra quelli ottenuti nei singoli chunk. Questa scelta riflette la semantica **multi-label** del problema: una categoria può manifestarsi anche in una sola porzione di una diff estesa ed, in quel caso, è comunque presente e va segnalata. Tramite **max**, basta che un solo chunk la rilevi con punteggio alto perchè essa emerga nel vettore finale. La media, al contrario, altera considerevolmente il punteggio, data la presenza degli zeri prodotti dai chunk in cui la categoria non compare, abbassando quella specifica categoria sotto la soglia di rilevazione e trasformando quest'ultima se corretta in un falso negativo.

4.4 Comunicazione con modello

Il dialogo con il modello avviene via rete verso un servizio remoto, che viene gestito tramite l'utilizzo di una chiave di accesso ad Ollama. Tale chiave viene oscurata nei log, per non esporre informazioni riservate. Essa impone un numero massimo di richieste e può restituire errori temporanei. Il client ha perciò alcuni accorgimenti per rendere la comunicazione affidabile. Come prima cosa, regola da solo la frequenza delle chiamate: tiene il conto delle richieste recenti e, se ci si avvicina al limite del servizio, attende quanto basta prima di inviare la successiva richiesta, evitando così di esser rifiutato.

Nel caso in cui il servizio segnala troppe richieste, il client aspetta, rispettando i tempi di attesa indicati dal servizio stesso, e riprova. Se l'errore è temporaneo, esso può essere causato o da un guasto momentaneo del server o da una piccola variazione casuale per non far ripartire tutte le richieste nello stesso istante. Se invece l'errore è definitivo, come una risposta illegibile, non effettua ulteriori tentativi.

Due accorgimenti applicati sono introdotti in questa fase per una restituzione corretta nei formati JSON desiderati: il primo accorgimento consiste in una regolazione preventiva della frequenza di invio. Il client mantiene la finestra temporale scorrevole entro cui registra gli istanti delle richieste recenti, determinandone il numero in tempo reale. Prima di ogni nuovo invio, questo conteggio viene confrontato con il limite imposto dal servizio: qualora la soglia risulta prossima alla saturazione, l'invio è posticipato affinché le richieste più datate escano dalla finestra di osservazione. Si ottiene così un distanziamento delle chiamate a priori, che mantiene la cadenza entro i limiti consentiti anziché reagire al rifiuto una volta intervenuto.

Il secondo accorgimento si occupa della gestione dei fallimenti residui ed è subordinato a una classificazione degli errori in base alla loro natura. In presenza di un superamento del limite di richieste, il client sospende l'esecuzione e ritenta in base all'intervallo di attesa eventualmente specificato dal servizio (*Retry-After*). In assenza di tale indicazione, l'attesa è determinata da una politica di **backoff esponenziale**, in cui l'intervallo tra tentativi successivi cresce esponenzialmente, così da concedere al servizio un margine di ripristino maggiore. All'intervallo si somma una componente stocastica (*jitter*): poiché più richieste possono fallire in concomitanza, la perturbazione casuale dei tempi di ripresa previene una loro ripartenza sincrona e il loro conseguente picco di carico. Il numero di tentativi è limitato superiormente da una soglia prefissata. Questa politica si applica ai soli errori transitori, come un malfunzionamento momentaneo del server o l'assenza di risposta entro un certo limite di tempo previsto, mentre in presenza di errori permanenti, come una risposta sintatticamente illeggibile, l'esecuzione è interrotta senza ulteriori tentativi.

4.5 Parsing

I modelli possono violare i limiti scelti di output: come scrittura in markdown, struttura troncata, punteggi come stringhe, ecc. Non prendere in considerazione queste risposte significherebbe perdere le classificazioni corrette; il parsing è realizzato appositamente come **graceful degradation**, ovvero una scala che parte dal rigoroso al permissivo, procedendo nel seguente modo:

1. un primo controllo viene effettuato sulle parentesi *string-aware* estraendo il primo oggetto JSON;
2. in caso di fallimento si tentano delle riparazioni mirate come l'inserimento di parentesi, virgole, chiusura di struttura;
3. si estraggono i punteggi finali via **regex**;
4. nella fase finale, i valori vengono normalizzati attenendosi al range $[0.0, 1.0]$, tollerando i numeri emessi come stringa.

Il medesimo trattamento è riservato a entrambi gli assi, poiché la risposta veicola in un unico oggetto sia il vettore delle categorie sia quello dei *maintenance type*.

Quando il modello produce un JSON valido ma effettua una classificazione errata, ad esempio azzerando tutto, interviene una **chain fallback** a più livelli, guidata dal principio

che nessuna diff con contenuto reale resta priva di etichette. Nel caso in cui si verifichi un chunk con vettore nullo o errori su una diff con righe modificate, vengono effettuati nuovi tentativi per un numero limitato di volte, conservando sempre il risultato migliore visto, un vettore con somma massima, saltando eventualmente i retry se esiste già una label con score superiore a 0.5. Il numero di tentativi predefinito è stato definito a **cinque** per evitare fallimento di scoring, essendo dipendente dalle ripetizioni dato che il modello può tendere a produrre errore.

Alla medesima ratio risponde l'inversione di due valori predefiniti. Il *retry* sulle diff la cui classificazione finale non presenti alcuna categoria oltre soglia (**flagged** = 0) è ora **attivo di default**, poiché un vettore che non supera mai la soglia, pur formalmente valido, è indistinguibile da un'astensione. L'*assistant-prefill*, che inietta una parentesi graffa aperta come ultimo messaggio dell'assistente affinché i modelli dotati di ragionamento emettano JSON visibile, è invece ora **disattivo** sul primo tentativo, essendo divenuto ridondante con l'output vincolato da schema; i tentativi successivi continuano a impiegarlo, dato che a quel punto la prima strategia si è già dimostrata insufficiente.

Ad ogni chunk e per ciascuna diff, viene assegnato uno **status** che può essere di quattro tipologie: **ok**, **recovered**, **retried_failed**, **fallback**. Poiché una diff estesa è classificata per chunk, lo status della diff propaga quello dei suoi chunk secondo il criterio della **severità massima**: basta che un solo chunk sia stato recuperato perché l'intera diff risulti **recovered**. Il criterio è deliberatamente pessimistico, poiché lo status serve a segnalare un dubbio, non a certificare un successo. Questa tassonomia consente, in fase di analisi, di distinguere le classificazioni genuine da quelle là dove il modello non è in grado di dare una classificazione appropriata.

4.6 Output

Il risultato è consolidato in due artefatti **JSONL**: un *record primario* completo (contenente categoria primaria, summary, vettore, flag con motivazione per giustificare le label scelte con punteggio più alto, stato, metadati di chunking) e un *record di valutazione* per le analisi a valle. La divisione in due file garantisce sia maggiore efficienza analitica che migliore tracciabilità.

Con l'introduzione del secondo asse, entrambi i record trasportano un blocco dedicato alla ragione della modifica: la foglia primaria (**primary_maintenance_type**), la classe genitore (**maintenance_class**), il *base type* con le due foglie **adaptive** ricomposte (**maintenance_base_type**), il vettore completo dei sei punteggi (**type_scores**) e la motivazione della foglia prevalente. Conservare il vettore integrale, e non la sola etichetta vincente, consente di riesaminare a posteriori il margine con cui una ragione ha prevalso sulle altre: una diff in cui **corrective** superi **perfective** di pochi centesimi documenta un'ambiguità che la sola etichetta finale occulterebbe.

La derivazione rispetta la separazione fra ciò che è chiesto al modello e ciò che è calcolato dal sistema, già impiegata per la soglia di flagging. Al modello è richiesto unicamente il

vettore dei punteggi; la foglia primaria è l'*argmax* del vettore aggregato sui chunk, mentre classe genitore e *base type* sono ricavati in codice dalle tabelle di lookup (Tabella 1). Poiché lo sdoppiamento di **adaptive** rende la mappatura una funzione totale della foglia, la classe genitore è esatta per costruzione: chiederla al modello, introdurrebbe una seconda fonte di verità, potenzialmente incoerente con la prima, senza alcun beneficio informativo.

Poiché un'esecuzione su migliaia di diff può durare ore, il sistema è progettato per essere resiliente alle interruzioni. La *ripresa* è *idempotente*: all'avvio si rileggono i **diff_id** già presenti nell'output e vengono saltati, mentre i nuovi record vengono aggiunti in coda (scrittura in *append*). I file prendono un nome con timestamp ordinabile lessicograficamente, così che l'ultima esecuzione sia sempre identificabile. Infine, **temperature** e **top-p** sono fissati esplicitamente per garantire la riproducibilità dei risultati.

La directory predefinita è quella convenzionale di progetto, risolta rispetto alla radice del repository e non alla *working directory* corrente, così che l'esecuzione dia il medesimo esito da qualunque posizione sia invocata. Essa risiede però in un'area sincronizzata da un servizio cloud, che può porre i file in lock durante la sincronizzazione e provocare errori di scrittura intermittenti in fase di *append*; il rimedio è l'opzione esplicita **--output-dir**, con cui indirizzare l'output verso un percorso non sincronizzato.

4.6.1 Analisi dell'output

Successivamente, è stata eseguita un'analisi per quantificare la distribuzione delle categorie che il modello LLM ha assegnato alle varie *diff* e per visualizzare potenziali bias o squilibri nelle valutazioni. I dati vengono valutati secondo due metriche di distribuzione:

- Distribuzione Primaria: si associa ad ogni record la categoria con il punteggio in assoluto più alto: $\max(scores, key = scores.get)$
- Distribuzione Generale: Data una certa soglia *threshold* vengono selezionate tutte le istanze dove una determinata categoria ottiene un punteggio pari o superiore ad essa.

La metrica di bias è essenziale per inferire le categorie che vengono identificate più spesso come secondarie. Essa consiste nella differenza percentuale tra la distribuzione generale e quella primaria: $DistribuzioneGenerale\% - DistribuzionePrimaria\%$

I grafici successivamente generati a partire dalle metriche sono i seguenti:

4.6.2 Distribuzione dei punteggi per categoria

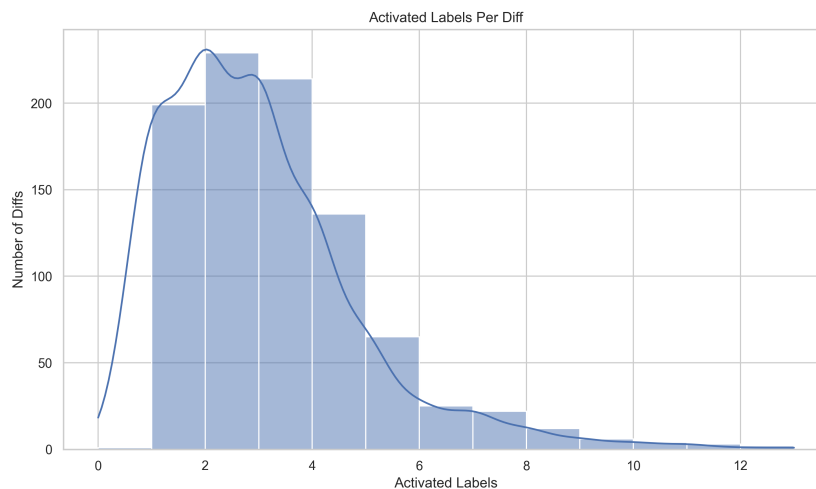


Figure 4: Distribuzione dei punteggi non nulli assegnati a ciascuna categoria.

Il boxplot fornisce una misura della confidenza del modello per ciascuna etichetta, condizionata ai casi in cui la categoria è stata effettivamente presa in considerazione. Le categorie *Testing*, *Build and Run*, *Impl. Details*, *Conf.&Env.*, *Documentation* e *System Overview* presentano le mediane più elevate, collocate attorno a 0,70, con scarti interquartili che si estendono fino a 0,85: quando il modello rileva questi pattern, li rileva con decisione, sintomo di un vocabolario e di direttive poco ambigue all'interno dei file markdown.

Di contro, *Performance* e *UI/UX* mostrano distribuzioni compresse verso il basso, con mediane che gravitano attorno a 0,30 e scarti interquartili che non raggiungono la soglia di flagging. L'andamento evidenzia un grado di incertezza maggiore da parte dell'LLM, che fatica ad assegnare loro score elevati; una difficoltà imputabile sia a una reale carenza di direttive robuste su questi temi negli ACF analizzati, sia a una sovrapposizione semantica con altre istruzioni generiche di progetto. E' interessante osservare che *AI Integration*, pur essendo la categoria dominante per frequenza, presenta una mediana intermedia (circa 0,50) su uno scarto interquartile molto ampio, che si estende da 0,30 a 0,85: la sua prevalenza nel corpus non deriva dunque da una confidenza uniformemente elevata, bensì dal fatto di essere pertinente a un numero molto grande di diff, alcune con evidenza forte e altre con evidenza debole.

4.6.3 Primario rispetto al generale

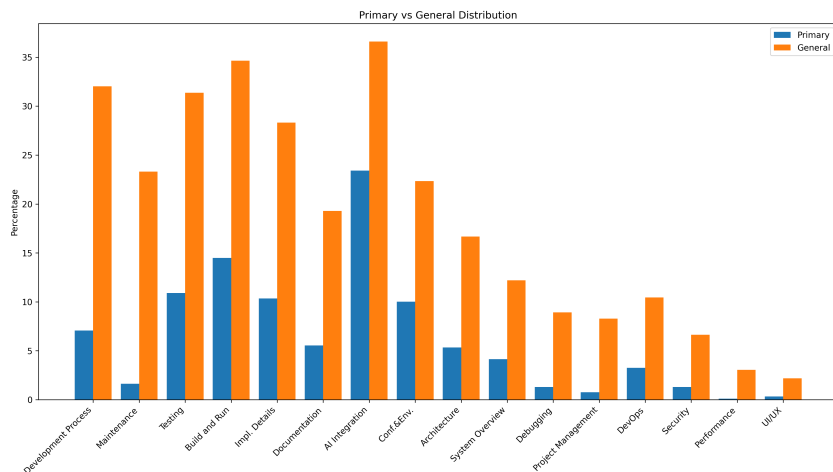


Figure 5: Confronto percentuale tra Distribuzione Primaria e Generale delle categorie.

A fronte del grafico a barre a confronto, emerge chiaramente come la categoria *AI Integration* domini l'intero output, rappresentando circa il 23.4% della distribuzione primaria e oltre il 36.6% di quella generale. Questo dato conferma che gran parte delle modifiche agli Agent Context Files (ACF) ruota esplicitamente attorno al raffinamento delle istruzioni comportamentali per i modelli.

Risulta altrettanto evidente il divario per categorie come *Development Process*, *Maintenance* e *Testing*: la prima passa dal 7,1% della distribuzione primaria al 32,0% di quella generale, e il caso della seconda è il più estremo dell'intero corpus, poiché *Maintenance* è categoria primaria in appena 15 diff (1,6%) ma compare oltre soglia in 214 (23,3%), ossia in quasi una diff su quattro. Ciò suggerisce che le direttive di manutenzione e quelle procedurali vengano modificate non in isolamento, ma in concomitanza con altre istruzioni principali. Al contrario, ambiti estremamente specifici come *UI/UX* (0,3% primaria, 2,2% generale) e *Performance* (0,1% primaria, 3,1% generale) risultano marginali in entrambe le distribuzioni.

4.6.4 Spostamento della distribuzione



Figure 6: Shift della distribuzione e analisi del Bias Delta percentuale.

Il grafico dello spostamento della distribuzione quantifica matematicamente il bias delle categorie verso ruoli secondari. Il *Net Change* evidenzia che *Development Process* con il +24.9%, *Testing* con il +20.5% e *Build and Run* con il +20.2% subiscono il maggior incremento percentuale tra le due distribuzioni. Questa metrica rivela una dinamica fondamentale nell'evoluzione degli ACF: queste etichette fungono da "categorie di supporto". Raramente la motivazione primaria di un commit è il solo aggiornamento di una regola di *Development Process*, eppure tale regola viene sistematicamente ritoccata o ribadita quando si aggiornano direttive più ampie (ad esempio, l'integrazione AI o l'architettura). Contrariamente, etichette come *UI/UX* con il +1.9% e *Performance* con il +2.9% mostrano uno shift quasi nullo, confermandosi come domini isolati che, quando presenti, costituiscono il focus esclusivo della diff.

4.6.5 Variazione della posizione in classifica

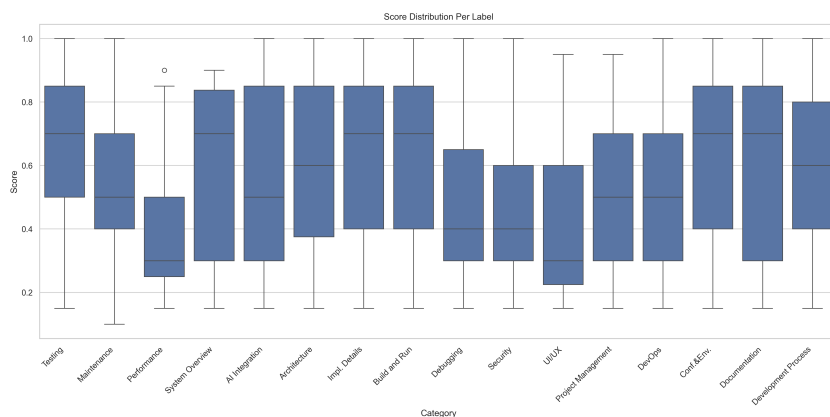


Figure 7: Variazione della posizione in classifica delle categorie tra classificazione Primaria e Generale.

L'analisi della varianza dei punteggi tramite boxplot fornisce una misura oggettiva della confidenza del modello LLM per ciascuna etichetta. Categorie come *Conf.&Env.*, *AI Integration* e *Architecture* presentano mediane elevate e intervalli interquartili ampi e verso la fascia alta. Questo indica una forte certezza del modello nel rilevarne i pattern semantici, sintomo di un vocabolario e di direttive poco ambigue all'interno dei file markdown. Di contro, categorie come *UI/UX* e *Performance* mostrano distribuzioni severamente compresse verso il basso, con mediane che gravitano attorno a 0.3. Questo andamento evidenzia un grado di incertezza maggiore da parte dell'LLM, che fatica ad assegnare score elevati; una difficoltà imputabile sia a una reale carenza di direttive robuste su questi temi negli ACF analizzati, sia a una sovrapposizione semantica con altre istruzioni generiche di progetto.

4.6.6 Matrice di co-occorrenza

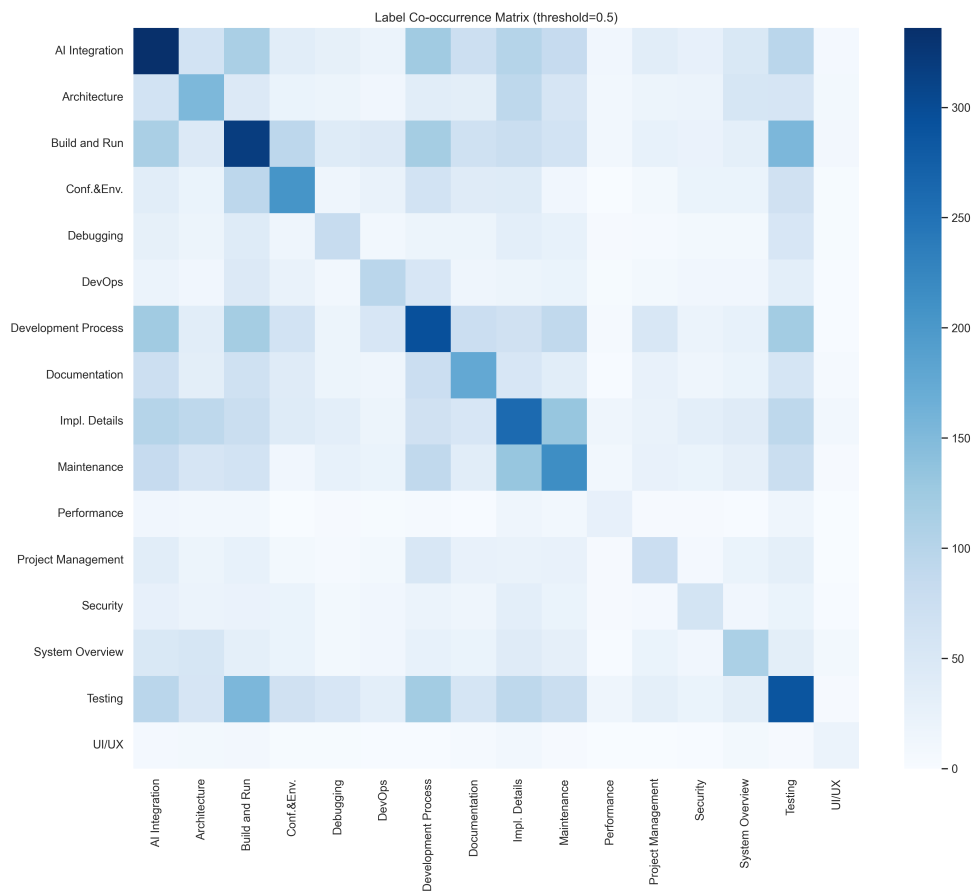


Figure 8: Matrice di co-occorrenza delle categorie all'interno delle stesse diff, alla soglia di attivazione di 0,50.

La matrice di co-occorrenza esplora le relazioni congiunte fra i topic all'interno della medesima diff, fissando la soglia di attivazione a 0,50. La densità di calore evidenzia raggruppamenti operativi molto chiari: la coppia più marcata al di fuori della diagonale è *Testing–Build and Run*, seguita da *Impl.Details–Maintenance*, *AI Integration–Development Process* e *Development Process–Build and Run*.

Questo fenomeno denota che gli sviluppatori approcciano l'aggiornamento dei context file in maniera olistica: quando raffinano il comportamento decisionale dell'agente (*AI Integration*) tendono simultaneamente ad aggiornare le regole procedurali (*Development Process*) e i comandi per eseguire e verificare il codice (*Build and Run*, *Testing*). Ciò conferma l'interdipendenza strutturale delle informazioni necessarie all'AI per operare in autonomia. La lettura della matrice richiede però una cautela: i conteggi assoluti sono dominati dalla frequenza marginale delle categorie, sicchè le celle chiare in corrispondenza di *Performance* e *UI/UX* riflettono in primo luogo la loro rarità e non l'assenza di una relazione. E' questa asimmetria a motivare la normalizzazione introdotta nella sottosezione seguente.

4.6.7 Probabilità condizionata

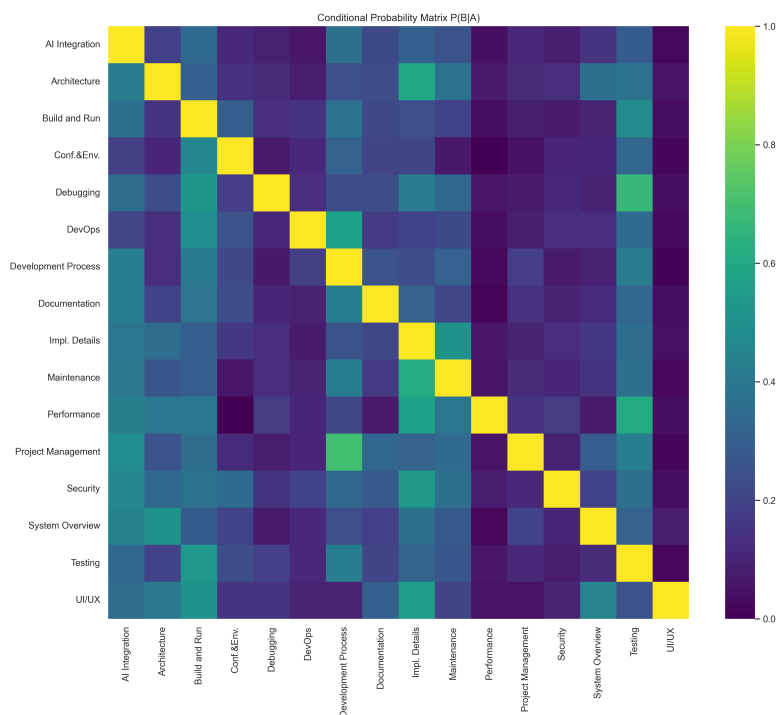


Figure 9: Probabilità condizionata tra le coppie di categorie $P(A|B)$.

Mentre la co-occorrenza indica la frequenza assoluta di combinazione, la matrice di probabilità condizionata $P(B|A)$ permette di far emergere relazioni asimmetriche di dipendenza direzionale fra i topic, poichè normalizza rispetto alla frequenza della categoria condizionante. Si osserva chiaramente come alcune categorie fungano da attivatori semantici per altre.

Il legame più forte dell'intera matrice è $P(\text{Development Process} | \text{Project Management})$, che si colloca attorno a 0,75: una diff che tocchi la gestione del progetto reca quasi sempre anche le regole procedurali che ne discendono. Seguono $P(\text{Impl. Details} | \text{Architecture})$ e $P(\text{Testing} | \text{Debugging})$, entrambe attorno a 0,65, e $P(\text{Impl. Details} | \text{Maintenance})$ attorno a 0,60. Anche la relazione fra *Testing* e *Build and Run* si conferma, con $P(\text{Build and Run} | \text{Testing})$ prossima a 0,55. Ciò dimostra che la stesura degli ACF segue precise catene logiche: la definizione di un ambiente di test o la risoluzione di bug impone all'autore di fornire all'agente anche l'impalcatura dei comandi di esecuzione e i vincoli di sviluppo necessari per completare il task.

L'asimmetria è essa stessa informativa. Il caso di *Debugging* e *Testing* ne è l'esempio più netto: la probabilità di osservare direttive di test data la presenza di direttive di debug è elevata, mentre la relazione inversa è sensibilmente più debole. Il debug degli ACF presuppone dunque il test, ma non viceversa, e una matrice simmetrica di co-occorrenza non avrebbe potuto documentare questa direzionalità.

4.6.8 Analisi dello status

Ulteriormente, è stato effettuato un controllo per ogni diff sulle caratteristiche di *status* dato che può assumere vari valori:

- *ok*: se l'intervento del LLM è andato a buon fine e senza errori;
- *recovered*: se sono presenti degli errori, si cerca comunque di ritornare un vettore valido con qualche *signal*;
- *retried_failed*: se il modello prova ma comunque il modello non riesce a ritornare un JSON valido o un *signal* usabile;
- *fallback*: nel caso in cui il modello ritorni un vettore a zero su una sezione di diff che ha effettuato cambiamenti;
- *else*: quando l'errore non rientra in queste categorie.

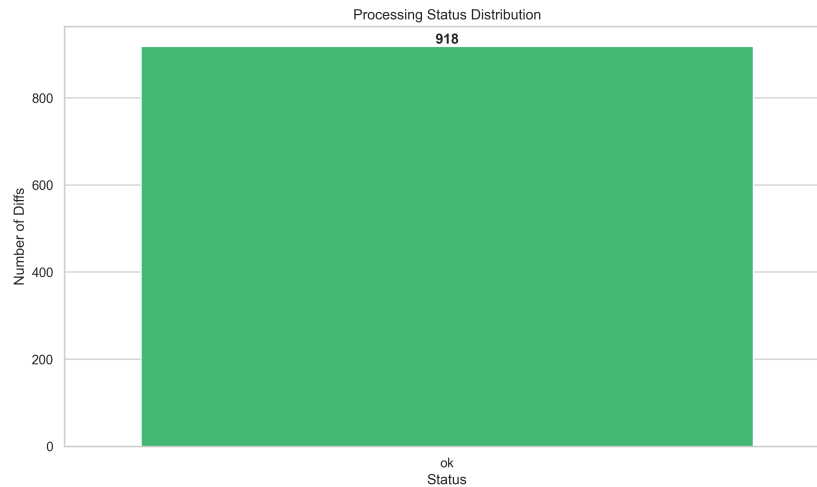


Figure 10: Conteggio numerico a barre della quantità di status per ciascuna tipologia.

Il grafico a barre illustra la distribuzione assoluta degli stati di elaborazione assegnati dalla pipeline. Su un totale di 918 diff processate, la totalità ha registrato uno status **ok**: l'istogramma presenta un'unica barra e nessuna diff è ricaduta nelle categorie **recovered**, **retried_failed** o **fallback**. Il tasso di successo netto si attesta dunque al 100%, come riportato dal grafico.

Processing Status Proportion

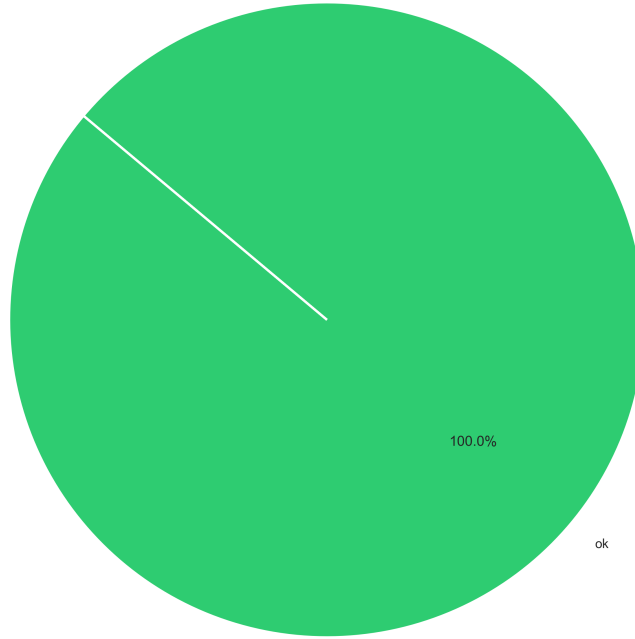


Figure 11: Grafico a torta degli status in rapporto percentuale tra le tipologie.

Come evidenziato dal grafico a torta, il tasso di successo netto, il `ok`, si attesta al 100%. Questa metrica conferma empiricamente l'elevata robustezza dell'architettura di comunicazione e parsing. Le strategie di *graceful degradation* adottate, come lo scanner string-aware e i tentativi di riparazione mirata dei JSON, hanno limitato il tasso di errore residuo allo 0.5% cumulativo. Di conseguenza, si può inferire che il dataset etichettato generato da questa esecuzione è strutturalmente integro e fornisce una base dati altamente affidabile per le successive fasi di *Cross-Model Validation*.

4.6.9 Distribuzione dei maintenance type

L'analisi condotta finora ha riguardato il solo asse delle categorie, ovvero l'*oggetto* delle modifiche. Poichè il modello produce in un'unica risposta anche il vettore dei *maintenance type*, è possibile caratterizzare le medesime 918 diff lungo l'asse della *motivazione*, verificando se le dinamiche osservate sui topic trovino un corrispettivo nelle ragioni che hanno prodotto le modifiche.

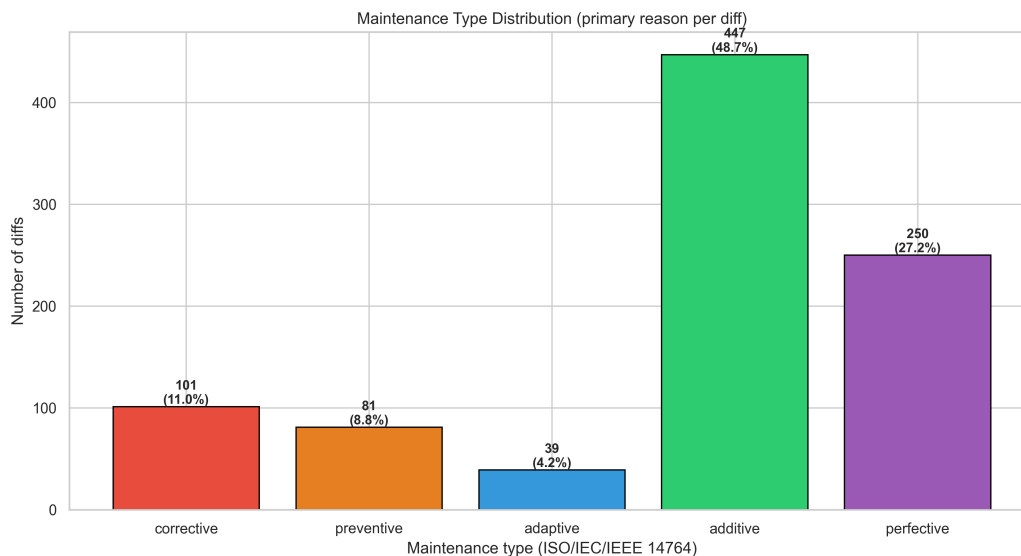


Figure 12: Distribuzione della foglia primaria (`primary_maintenance_type`) per le 918 diff, riportata sui cinque *base type* della Tabella 1.

Il grafico riporta, per ciascuna diff, la foglia che ha ottenuto il punteggio massimo, ricomposta sui cinque *base type*: le due foglie **adaptive** (**correction**) e **adaptive** (**enhancement**) sono qui riunite, coerentemente con il criterio di reporting descritto nella sezione di Prompt Engineering. La distribuzione è marcatamente sbilanciata: **additive** raccoglie da solo 447 diff (48,7%) e **perfective** altre 250 (27,2%), sicchè poco più di tre quarti delle modifiche agli ACF consistono nell'introduzione di direttive nuove o nella riformulazione di direttive già valide. Le motivazioni riconducibili a un malfunzionamento sono nettamente minoritarie: **corrective** si ferma a 101 occorrenze (11,0%), **preventive** a 81 (8,8%) e **adaptive** a 39 (4,2%).

Il dato è significativo se letto congiuntamente all'evidenza della evoluzione, dove la netta prevalenza dello status **modified** mostrava che gli ACF sono artefatti sottoposti a revisione continua. Quella statistica documentava la *frequenza* della manutenzione ma non la sua *natura*: l'asse introdotto qui la qualifica, mostrando che la revisione continua non è sintomo di instabilità dell'artefatto. Un file ripetutamente riparato e uno progressivamente arricchito esibiscono lo stesso tasso di modifica, e la distribuzione colloca il corpus analizzato sul secondo dei due regimi: l'ACF cresce per accumulo, e la riparazione è l'eccezione.

La scarsità di **adaptive** merita una lettura prudente. L'adeguamento a un ambiente mutato, quale l'aggiornamento di un toolchain o il cambio dell'agente di riferimento, presuppone un orizzonte temporale che la finestra di commit adottata nella sezione di estrazione osserva solo parzialmente: il valore del 4,2% va dunque inteso come una stima per difetto, non come una misura della rarità del fenomeno nella vita completa di un ACF.

4.6.10 Correction rispetto a Enhancement

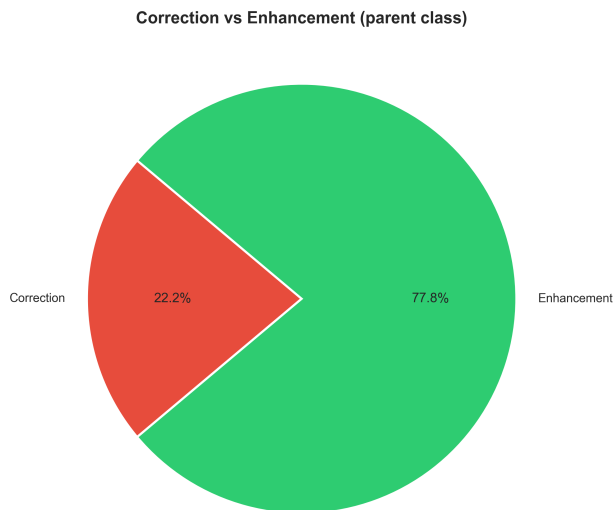


Figure 13: Ripartizione delle diff sulla classe genitore della *Modification Request* ISO/IEC/IEEE 14764:2022, derivata in codice dalla foglia primaria.

Il grafico aggrega le foglie sul primo livello della tassonomia normativa. Il rapporto è di circa 3,5 a 1 in favore di **Enhancement**, che copre il 77,8% delle diff (714) contro il 22,2% di **Correction** (204). Poichè la distinzione fra le due classi non riguarda la dimensione dell'intervento ma il suo rapporto con i requisiti, il dato afferma che gli sviluppatori impiegano gli ACF prevalentemente per *estendere* il contesto fornito all'agente, e solo in un caso su quattro e mezzo per ripararlo rispetto a un'aspettativa disattesa.

E' opportuno osservare che questa figura non è un secondo giudizio del modello, bensì una funzione totale della foglia primaria: la ripartizione è ottenuta applicando la tabella di lookup della Tabella 1, ed è proprio lo sdoppiamento del tipo *dual-parented* a renderla esatta per costruzione. La sua utilità è visibile nella scomposizione dei due settori: il 22,2% di Correction si compone di 101 diff **corrective**, 81 **preventive** e 22 **adaptive (correction)**, mentre il 77,8% di Enhancement si compone di 447 **additive**, 250 **perfective** e 17 **adaptive (enhancement)**. Le 39 diff **adaptive** si distribuiscono dunque quasi equamente fra le due classi (22 contro 17): esse sarebbero state, in una tassonomia non disambiguata, il residuo che avrebbe reso arbitraria l'attribuzione del primo livello, e la loro ripartizione bilanciata conferma a posteriori che il criterio temporale introdotto nel prompt discrimina un'ambiguità reale.

4.6.11 Distribuzione dei punteggi per maintenance type

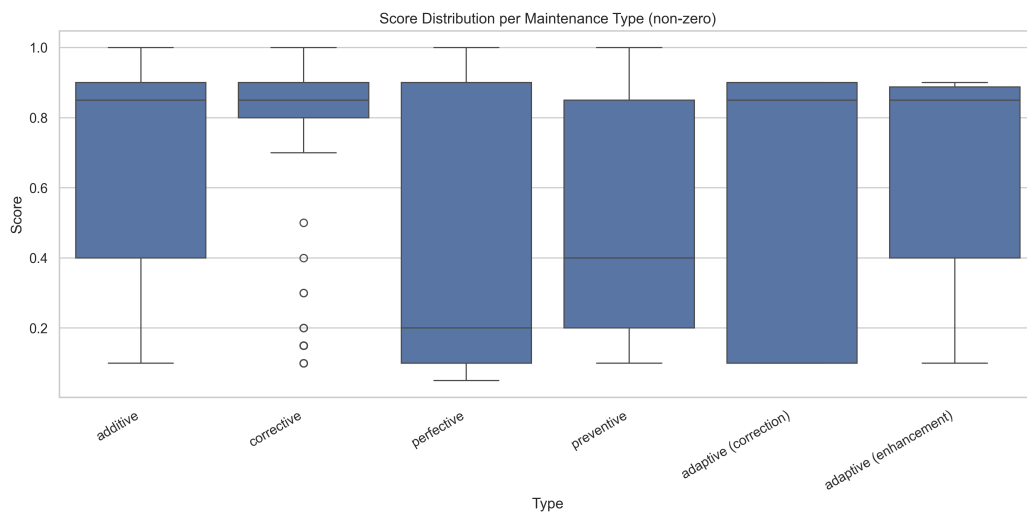


Figure 14: Distribuzione dei punteggi non nulli per ciascuna delle sei foglie della tassonomia. Il condizionamento sui valori non nulli isola la confidenza del modello nei casi in cui la foglia è stata effettivamente presa in considerazione.

Il grafico riporta, per ciascuna delle sei foglie, la distribuzione dei punteggi ristretta ai valori non nulli; le due foglie **adaptive** sono qui mantenute distinte, essendo il vettore prodotto dal modello a sei componenti. Il condizionamento è necessario: includere gli zeri, che costituiscono la maggioranza dei valori su un asse quasi mutualmente esclusivo, comprimerebbe ogni distribuzione verso il basso e misurerebbe la rarità della foglia anziché la confidenza con cui viene assegnata.

Il grafico permette di verificare empiricamente il vincolo di quasi-mutua-esclusività comunicato al modello. Le mediane si collocano attorno a 0,85 per **additive**, **corrective**, **adaptive (correction)** e **adaptive (enhancement)**, ben oltre la fascia alta ($\geq 0,60$) prescritta dal prompt per il tipo dominante: quando una di queste foglie viene attivata, essa lo è come motivazione prevalente e non come sfumatura. Il comportamento di **corrective** è il più netto, con uno scarto interquartile compresso fra 0,80 e 0,90 e i baffi che non scendono sotto 0,70; la coda di valori anomali fra 0,10 e 0,50 rappresenta i casi in cui la riparazione è stata considerata e scartata, ossia il confine insidioso fra **corrective** e **perfective**.

Speculare è il profilo di **perfective**, la cui mediana si colloca a 0,20 pur con uno scarto interquartile che si estende fino a 0,90: la distribuzione è bimodale e la foglia opera in due regimi distinti. Nelle 250 diff in cui prevale, essa è assegnata con punteggio elevato; nelle restanti figura come punteggio residuo, ovvero come l'ipotesi che il modello mantiene aperta ogni volta che una modifica tocca la forma di una direttiva senza alterarne la sostanza. Un andamento affine, con mediana a 0,40, caratterizza **preventive**.

4.6.12 Traiettoria di manutenzione di un singolo ACF

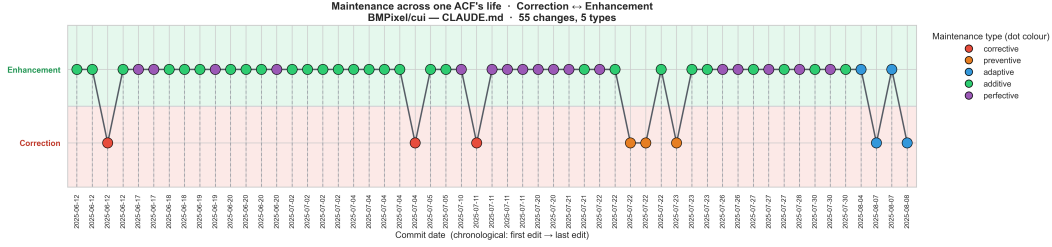


Figure 15: Traiettoria delle motivazioni di manutenzione lungo la vita del file `CLAUDE.md` della repository `BMPixel/cui`: 55 modifiche ordinate cronologicamente, dal 12 giugno all'8 agosto 2025. L'ordinata riporta la classe genitore, il colore del punto la foglia.

Le distribuzioni aggregate finora presentate descrivono il corpus nel suo insieme, ma appiattiscono la dimensione temporale: una motivazione è un evento situato nella storia di un file, e la sua posizione in tale storia è parte del suo significato. Il seguente grafico restituisce questa dimensione seguendo un singolo artefatto, il `CLAUDE.md` di `BMPixel/cui`, lungo le sue 55 modifiche: l'ascissa scorre cronologicamente dalla prima all'ultima revisione, l'ordinata distingue le due classi genitore e il colore identifica la foglia, cosicché le due granularità della tassonomia siano leggibili simultaneamente. Si tratta, va precisato, di un *case study* illustrativo e non di un'evidenza statistica: la traiettoria di un singolo file non è generalizzabile al corpus, ed è presentata per mostrare che cosa l'asse di manutenzione renda osservabile a livello di artefatto.

La traiettoria conferma su scala locale il regime di crescita per accumulo osservato in aggregato. Il tracciato permane stabilmente nella banda Enhancement, alternando **additive** e **perfective** in lunghe sequenze ininterrotte, e discende verso la banda Correction solo in episodi isolati e di breve durata. Tali discese non sono però distribuite in modo uniforme, e la loro composizione muta nel tempo: le prime, fra giugno e l'11 luglio, sono **corrective** isolate, coerenti con la messa a punto di un file di recente introduzione; il 22 e 23 luglio compare invece un addensamento di **preventive**, che documenta un intervento deliberato sulle direttive prima che queste si rivelino difettose; le ultime, il 4 e l'8 agosto, sono **adaptive**.

Proprio queste ultime illustrano il guadagno informativo dello sdoppiamento della foglia *dual-parented*: il 4 agosto due modifiche adiacenti recano il medesimo colore ma si collocano su bande opposte, l'una come adeguamento proattivo a un ambiente in via di mutamento e l'altra come reazione a un ambiente già mutato. Una tassonomia che avesse restituito il solo **adaptive** le avrebbe rese indistinguibili, e con esse la transizione da una fase di consolidamento del file a una di inseguimento del contesto esterno.

5 Valutazione delle ambiguità

La risposta di un modello può presentare incertezza dovuta sia a limiti del modello stesso che da un problema semantico presente all'interno di una diff. Al fine di garantire la validità delle

risposte del LLM e la robustezza delle sue predizioni, è stata introdotta una *Cross-Model Validation*. Il sistema esegue un campionamento sugli stessi record valutati dal modello primario, processando gli stessi input con tre identici LLM di fascia superiore, con maggiore capacità di ragionamento e di comprensione del contesto. Ad ogni diff valutato, vengono incrociati i risultati dei tre modelli per calcolare un tasso di accordo, un *Agreement Rate* per stabilire una *Ground Truth* dinamica.

L'architettura proposta prevede un'esecuzione parallela e asincrona di classificazione su tre LLM utilizzati in cloud, specificamente: `kimi-k2.7-code:cloud`, `qwen3.5:cloud`, `glm-5.2:cloud`. Sono stati utilizzati come dei valutatori indipendenti al fine di ottenere un tasso di accordo inter-modello.

5.1 Metriche di accordo Cross-Model

Questa analisi si articola su due livelli:

- **Single-label agreement:** focalizzato sulla categoria primaria `primary_category`, prevedere un'analisi percentuale di accordo totale tra tutti e tre i valutatori, la percentuale di accordo media *pairwise*, il *Kappa di Fleiss* per categorie nominali e il *Kappa di Cohen medio pairwise*.
- **Multi-label agreement:** Analizza i set di categorie che supera una certa soglia di punteggio, con valore default a 0.50. Vengono misurate le stesse metriche della single-label ma per singola categoria trattata in forma binaria nel caso in cui sia presente o assente.

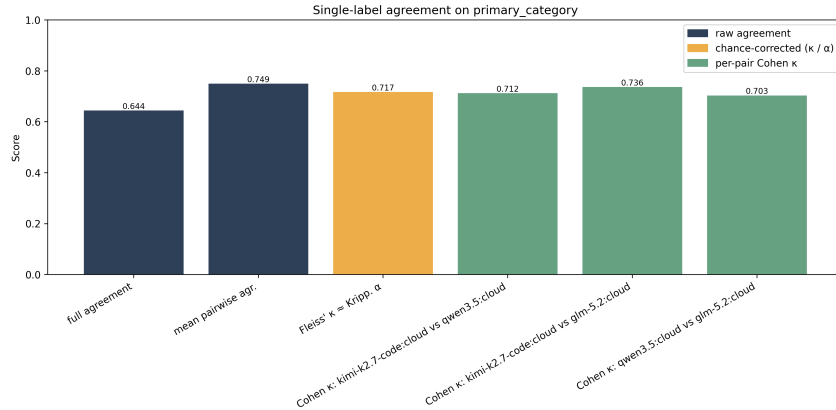


Figure 16: Confronto delle metriche di accordo single-label sulla primary category tra i modelli.

Come si evince dal grafico, l'accordo *single-label* sulla categoria primaria mostra risultati notevoli. Il *full agreement*, l'unanimità esatta tra i tre modelli, si attesta al 64.4%, mentre l'accordo medio *pairwise* sale al 74.9%. Le metriche corrette per il caso *Chance-corrected*, ovvero il Kappa di Fleiss e i vari Kappa di Cohen inter-modello, si raggruppano in un intervallo ristretto tra 0.703 e 0.736. Secondo la scala di Landis e Koch, questi valori indicano un accordo *Substantial*, dimostrando che l'estrazione del *topic* principale da parte degli LLM è un processo robusto e poco incline a divergenze o allucinazioni.

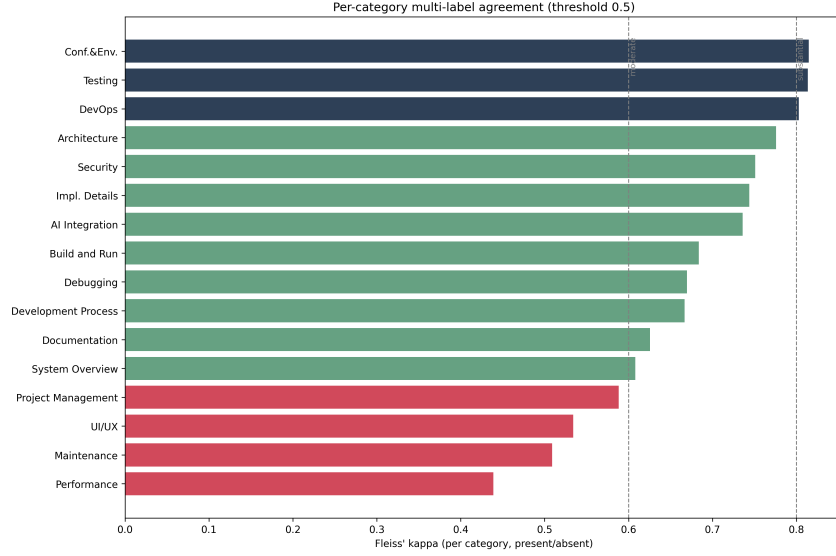


Figure 17: Accordo multi-label per singola categoria misurato tramite il Kappa di Fleiss.

Passando all'analisi *multi-label* per singola categoria, il Kappa di Fleiss evidenzia con precisione quali tematiche risultano più oggettive. Categorie fortemente tecniche e strutturate come *Conf.&Env.* e *Testing* superano la soglia dello 0.80, raggiungendo un accordo "Quasi Perfetto". La maggior parte delle categorie architettureali *DevOps*, *Architecture*, *Security* si colloca solidamente nella fascia di accordo sostanziale. Al contrario, categorie trasversali come *Performance* e *Maintenance* registrano i punteggi più bassi, suggerendo che i confini semantici di queste direttive all'interno dei context file sono maggiormente sfumati e aperti all'interpretazione soggettiva dei modelli.

5.2 Quantificazione dell'Ambiguità

L'ambiguità è quantificata a livello di singola istanza e aggregata per categoria, permettendo l'identificazione delle aree tematiche più soggette a divergenze:

- **Per istanza:** Viene calcolata l'entropia di Shannon sui tre voti primari espressi in bit e la frazione di disaccordo. Sul piano multi-label, si utilizza la distanza di Jaccard media *pairwise*.
- **Per categoria:** Si valuta il tasso di disaccordo sulla presenza, calcolato come la frazione di istanze in cui i valutatori non sono unanimi. Si calcola inoltre l'entropia media dei voti primari per le istanze in cui quella determinata categoria rappresenta la pluralità.

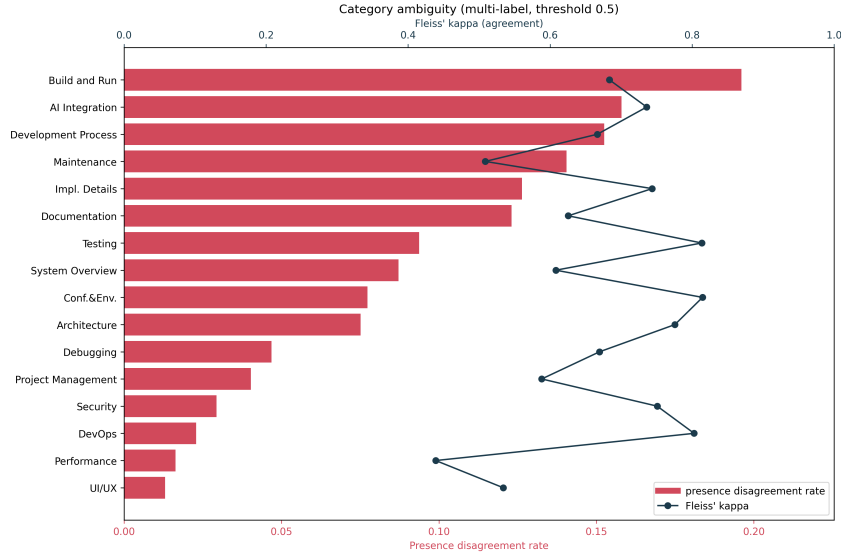


Figure 18: Ambiguità per categoria valutata tramite tasso di disaccordo sulla presenza e Kappa di Fleiss.

Il grafico proposto incrocia il tasso di disaccordo sulla presenza con il Kappa di Fleiss, rivelando una dinamica operativa fondamentale: categorie ad altissima frequenza come *Build and Run* e *AI Integration* presentano i tassi di disaccordo assoluti più alti, ma riescono a mantenere un Kappa di Fleiss solido e situato nella fascia di accordo sostanziale. Discorso profondamente diverso per *Maintenance*, che evidenzia una reale e marcata ambiguità semantica: a fronte di un disaccordo rilevante, il suo Kappa crolla in prossimità dello 0.52. Le direttive come *Performance* e *UI/UX* mostrano un disaccordo minimo quasi esclusivamente per via della loro scarsità statistica all'interno del dataset.

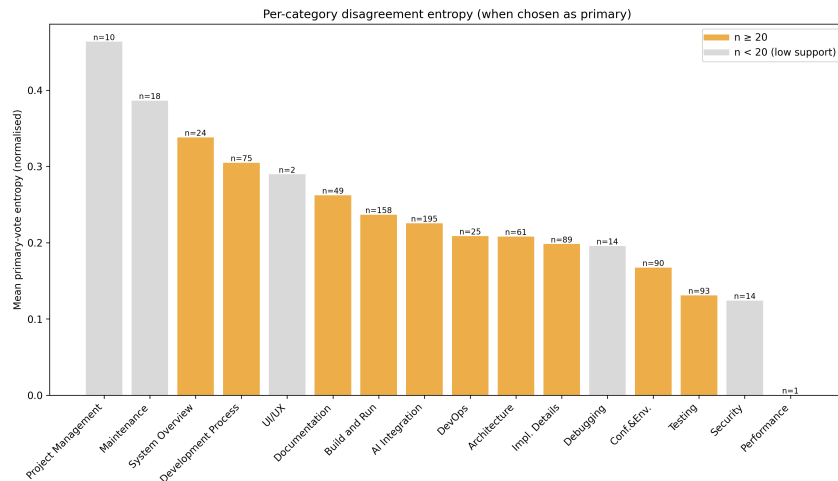


Figure 19: Entropia media dei voti primari aggregata per categoria.

L'entropia media per categoria illustrata fornisce un'ulteriore lente sull'incertezza decisionale in fase di classificazione primaria. L'istogramma separa le categorie ad alto e basso supporto per pesare correttamente i dati: *Project Management*, pur sfoggiando l'entropia

più alta in assoluto, si basa su un supporto statistico modesto. Focalizzandosi sul campione statisticamente significativo, si nota che *System Overview* e *Development Process* registrano i massimi livelli di entropia, indicando un’alta frammentazione dei voti tra i valutatori. Di converso, *AI Integration* e *Build and Run* mantengono un’entropia contenuta, confermando un eccellente livello di coerenza definitoria interna a dispetto dell’enorme mole di occorrenze.

5.3 Ambiguità dell’Asse di Manutenzione

La valutazione dell’accordo e dell’ambiguità è correlata in base anche alle direttive di manutenzione basate sullo standard ISO/IEC/IEEE 14764 [2].

- L’accordo su categoria primaria viene calcolato sulla base di due domini nominali distinti, con 2 classi genitore: *Correction* ed *Enhancement*; e 5 tipi base: *corrective*, *preventive*, *perfective*, *additive*, *adaptive (correction)*, *adaptive (enhancement)*;
- Per identificare quale ragione stia introducendo maggiore incertezza, viene estratto un tasso di disaccordo e un *Kappa di Fleiss* binario per ciascuna tipologia di manutenzione.

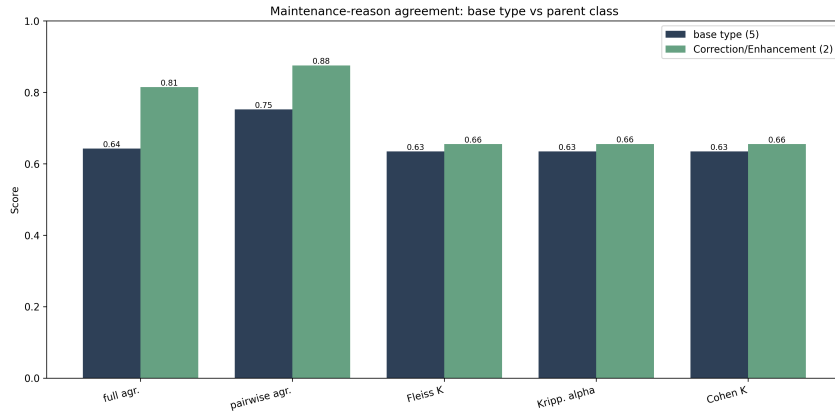


Figure 20: Accordo inter-modello sulla classificazione dei Maintenance Type: classi genitore vs tipologie di base.

Il grafico quantifica il divario prestazionale tra la classificazione a risoluzione fine e quella ad alto livello. Come ci si aspetterebbe, l’astrazione alla macro-intenzione *Correction* vs *Enhancement* mitiga l’incertezza semantica: l’accordo unanime *full agreement* passa dal 64% all’81% e l’accordo *pairwise* sfiora l’88%. Eppure, analizzando le metriche corrette per il caso, si osserva un incremento decisamente più conservativo. Questo scarto dimostra che per gli LLM è relativamente facile determinare la polarità macroscopica di una modifica, ma resta complesso discernere il confine sottile che separa tipologie evolutive affini all’interno dello stesso ramo, come un intervento puramente additivo rispetto ad uno di perfezionamento.

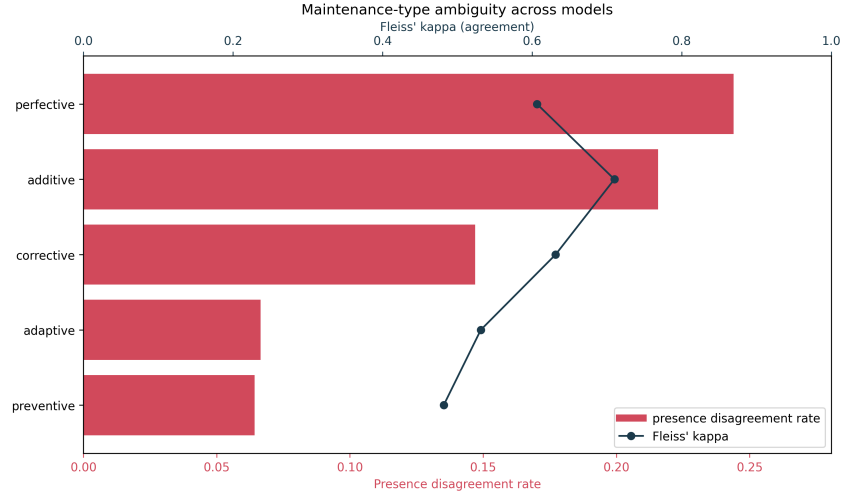


Figure 21: Livello di ambiguità rilevato per ciascuna tipologia di manutenzione standard.

Scendendo nel dettaglio dell'asse motivazionale, il grafico seguente evidenzia come le direttive *perfective* e *additive* siano l'oggetto primario di dibattito tra i valutatori, accumulando tassi di disaccordo rispettivamente superiori al 24% e al 21%. A parità di elevato tasso di disaccordo, la resilienza semantica differisce enormemente: *additive* vanta comunque un Kappa robusto, mentre *perfective* risulta frammentato e fragile. Tuttavia, l'anomalia più critica risiede nella categoria *preventive*: seppur meno ricorrente nel dataset, registra il livello di accordo *Kappa* più basso dell'intera tassonomia. Questo espone la difficoltà insita nell'estrarre deduzioni proattive dal solo testo, confondendo facilmente una prevenzione tecnica con un generico aggiornamento architettuale.

5.4 Confronto con il Modello di Riferimento

Per validare ulteriormente i risultati, le predizioni di ogni modello vengono confrontate individualmente con un LLM primario di riferimento, **gemma4:31b-cloud**. È possibile misurare la deviazione di ciascun LLM a confronto con la baseline del modello di riferimento per le solo diff valutate in comune. Questo perché le chiamate dei modelli possono fallire su diff differenti, quindi eventuali errori sono stati scartati.

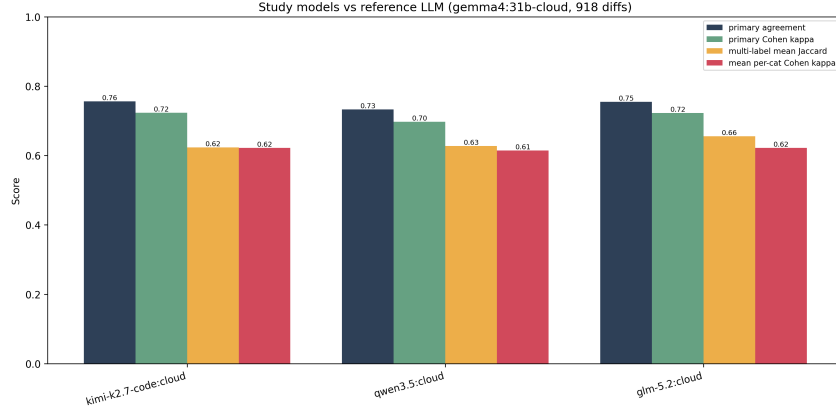


Figure 22: Confronto delle metriche di accordo e scostamento dei modelli di studio rispetto all'LLM di riferimento.

Il grafico certifica il ridotto scostamento tra i tre modelli *peer* di studio e la robusta baseline costituita dal modello **gemma4:31b-cloud**. L'accordo isolato sulla categoria primaria oscilla stabilmente per tutti tra il 73% e il 76%, supportato da un Kappa di Cohen uniformemente posizionato al di sopra della soglia dello 0.70 *Substantial Agreement*. Nel confronto, il modello **glm-5.2:cloud** si distingue per una marginale ma costante superiorità, trainando l'allineamento specialmente sul delicato versante multi-label, dove tocca un Jaccard medio di 0.66.

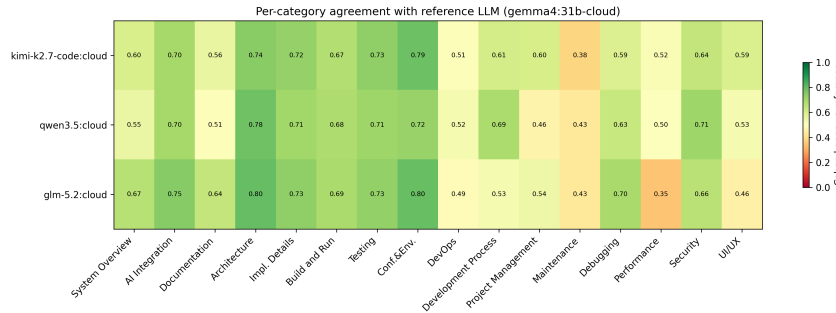


Figure 23: Mappa di calore (Heatmap) del Kappa di Cohen per singola categoria valutato in raffronto al modello di riferimento.

La mappa di calore fornisce la verifica granulare e definitiva sui pattern di ambiguità intrinseca discussi in precedenza. Le direttive esecutive ad alto livello di strutturazione formale *Architecture*, *Testing*, *Conf.&Env.* mostrano blocchi uniformemente verdi e concordanza pressoché totale con il modello primario. Diametralmente opposta la situazione per domini metodologici sfumati, in primis *Maintenance* e *Performance*, che restituiscono colonne calde e disallineate, spingendosi a cali critici. L'omogeneità verticale di questi deficit dimostra che l'incertezza semantica risiede nella complessità della diff stessa e non in un limite architettonico del singolo LLM.

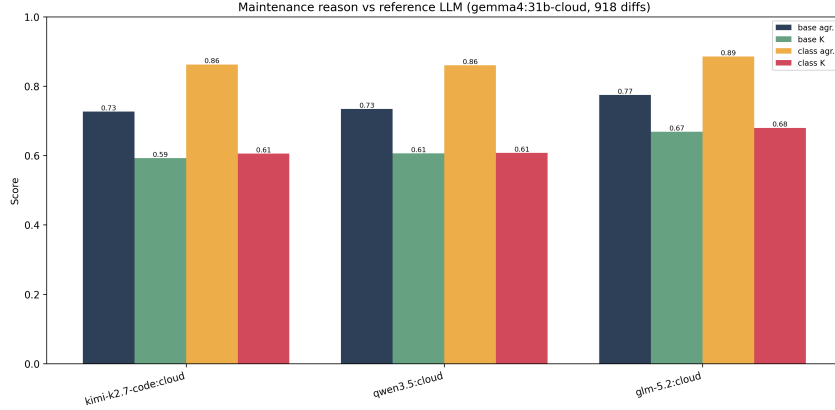


Figure 24: Confronto delle prestazioni di accordo sui Maintenance Type rispetto al modello LLM primario.

Confrontando le performance sull'asse della manutenzione, questo grafico conferma che i modelli di studio emulano brillantemente le reti concettuali del modello primario. L'identificazione della macro-classe genitore si attesta su percentuali vertiginose, culminando nell'89% di accordo netto per la controparte `glm-5.2:cloud`. Imporre la più esigente classificazione fine basata sui 5 tipi base comporta l'attesa flessione prestazionale, che si stabilizza comunque in un *range* virtuoso compreso tra il 73% e il 77% di *base agreement*. Trattandosi di *Agent Context File* densi di ambiguità discorsive e natural language, tali valori rappresentano un allineamento d'eccellenza.

5.5 Accordo aggregato

Un ulteriore statistica puramente descrittiva è stata calcolata come *Pooled Agreement*, ovvero il modello di riferimento viene inserito insieme ai tre LLM considerando ciascun modello come un valutatore intercambiabile sull'intera risposta multi-label.

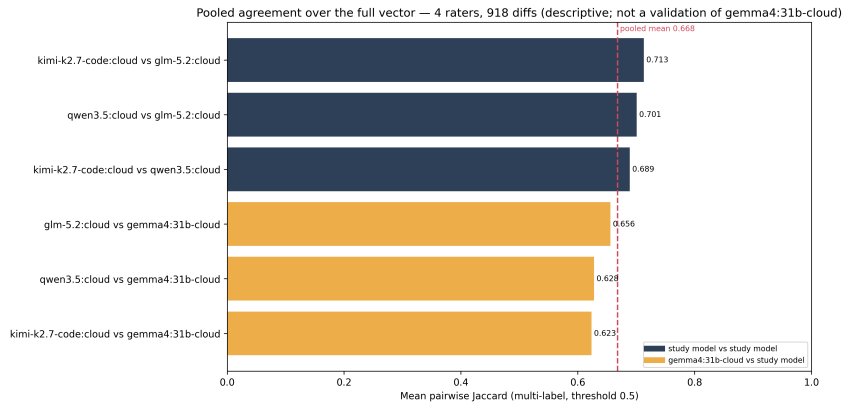


Figure 25: Accordo aggregato (Pooled Agreement) sull'intero vettore multi-label, calcolato includendo il modello di riferimento nel pool dei valutatori.

La metrica *Pooled Agreement* mappa la distanza di Jaccard media calcolata per ogni incrocio possibile nel consorzio dei quattro modelli. Dalla visualizzazione a barre emerge un *bias*

strutturale che interessa le coppie interamente formate dai modelli di studio, es. `klm` vs `glm`, mostrano una densità di convergenza maggiore, con tassi tra 0.689 e 0.713, rispetto alle combinazioni che introducono la *baseline* a 31b. Questa polarizzazione lascia supporre che i modelli di calibro inferiore condividano logiche di astrazione affini, differenziandosi in minima parte dall'iper-generalizzazione garantita da un'architettura più imponente. Nonostante tale scarto, la media aggregata inter-pool *pooled mean* si posiziona stabilmente sul valore 0.668 come si evince dalla linea rossa, convalidando appieno l'affidabilità semantica e l'efficacia trasversale del sistema di validazione Multi-LLM.

6 Golden Standard

Per verificare ulteriormente la correttezza delle predizioni prodotte dagli LLM, è stato costruito un golden standard come riferimento di verità assoluta. Il Golden Standard è stato effettuato prendendo in esame di 980 diff di cui si compone il dataset, 150 diff selezionate in modo da riflettere fedelmente la composizione tematica dell'intero corpus. La scelta non è stata casuale: dato che ciascuna diff porta in media 2,9 categorie sopra soglia, un semplice campionamento uniforme avrebbe finito per privilegiare le categorie più frequenti (come AI Integration e Build and Run) lasciando invece scoperte quelle marginali. Per evitare questo sbilanciamento, la selezione è stata articolata in due momenti:

- **Fase di Censimento:** In primo luogo, si è deciso di includere per intero tutte le diff appartenenti alle categorie più rare (nello specifico UI/UX e Performance, entrambe con frequenza primaria inferiore all'1%) poiché una selezione probabilistica su un supporto così sottile avrebbe prodotto stime inaffidabili.
- **Fase di Stratificazione Iterativa:** In secondo luogo, sulle diff restanti è stato applicato un algoritmo di stratificazione iterativa (Sechidis et al., 2011) che, procedendo un elemento alla volta, seleziona di volta in volta la diff capace di riequilibrare al meglio le proporzioni delle sedici categorie, dando priorità a quelle ancora sottorappresentate. La procedura si arresta al raggiungimento delle 150 osservazioni complessive, dimensione scelta come compromesso fra la sostenibilità dello sforzo di annotazione manuale e la significatività statistica dei risultati.

Successivamente a quest'analisi, è stato costruito il file excel contenente le seguenti informazioni:

- **diff_id:** chiave stabile della diff, costruita concatenando l'hash del commit di riferimento e il nome del file (es. `c200ab20...:CLAUDE.md`). Serve come identificatore univoco per la ricongiunzione con le predizioni dei modelli nel file chiave.
- **filename:** nome dell'Agent Context File analizzato (tipicamente `CLAUDE.md` o `AGENTS.md`), fornito come contesto di ambientazione della modifica.
- **commit_url:** link diretto alla diff del file all'interno del commit su GitHub, ancorato a `#diff-<sha256(path)>` in modo che all'apertura del link, si viene reindirizzati direttamente al riquadro del file annotato e non al vertice del commit.

- **commit_date:** timestamp del commit in formato ISO-8601, necessario a collocare temporalmente la modifica.
- **commit_message:** messaggio di commit integrale associato alla modifica.
- **prev_change_1, prev_change_2, prev_change_3:** le tre modifiche precedenti dello stesso file, ordinate cronologicamente dalla più remota alla più recente e rese come stringa data. Ognuna di esse è stata rappresentata sottoforma di link cliccabile che riporta alla diff del rispettivo commit su GitHub.
- **changed_lines:** corpo effettivo della modifica: righe aggiunte e rimosse dello unified diff, ripulite dei marker strutturali (+++, —, @@) e ripresentate una per riga con prefisso + o -.
- **labels_gold:** Colonna aggiuntiva vuota, successivamente riempita con l'insieme di tutte le categorie tematiche applicabili alla diff. Le sedici etichette ammissibili sono quelle della tassonomia definita in categories.json.
- **maintenance_gold:** Colonna aggiuntiva vuota, nella quale è stato integrato il singolo maintenance type scelto fra corrective, preventive, adaptive (per il quale viene specificata la classe padre tra **Correction** e **Enhancement**), additive, perfective.

Per ciascuna riga, sono state esaminate in sequenza le righe modificate (colonna changed_lines) per identificare l'insieme delle categorie tematiche pertinenti e successivamente annotate in labels_gold, applicando le medesime sedici definizioni della tassonomia fornite al modello nel file categories.json. Successivamente, per determinare il maintenance type (indicato nel file excell con maintenance_gold), sono stati integrati le seguenti informazioni:

- Il messaggio del commit corrente.
- Il contenuto rimosso all'interno delle righe modificate (per distinguere corrective da perfective sulla base della correttezza dello stato precedente)
- La traiettoria delle tre modifiche precedenti, cliccando sui link prev_change_* quando la sola diff non era sufficiente a discriminare

Una volta conclusa questa sezione, lo step successivo consiste nell'effettuare il confronto tra le predizioni ottenute dagli LLM con il Golden Standard calcolato manualmente sia per le Categorie che per la Maintenance attraverso le metriche Accuracy, Precision, Recall ed F1-Score. Per una maggiore osservabilità delle performance, si è deciso di effettuare due analisi separate delle due classi in questione.

6.1 Confronto tra risultati ottenuti tramite gli LLM con Golden Standard per Categoria

Il grafico seguente, riporta per ciascuno dei quattro modelli, le quattro metriche aggregate sull'asse multi-label rispetto al golden standard: Precision, Recall, micro F1 e Accuracy. L'accuratezza a livello di decisione binaria si attesta stabilmente attorno all'85% per tutti e

quattro i modelli. Per quanto riguarda le altre tre metriche si collocano invece nella fascia 0,56–0,80.

I tre modelli di studio (glm-5.2, kimi-k2.7-code, qwen3.5) mostrano un comportamento molto simile tra loro: la Precision (0,66–0,70) supera sistematicamente la Recall (0,57–0,61), mentre per quanto riguarda il modello primario Gemma4:31b-cloud, registra una Precision più bassa (0,60) ma allo stesso tempo una Recall di gran lunga più alta (0,80). Di conseguenza anche l’F1-Score di Gemma risulta più elevato (0,68).

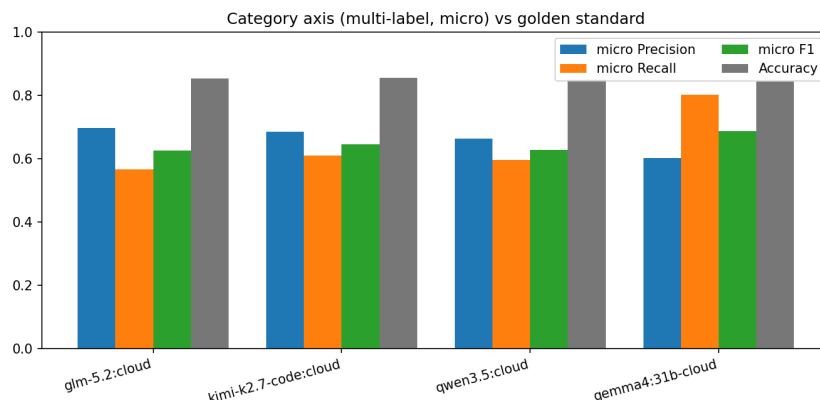


Figure 26: Confronto tra Golden Standard e Multi-LLM e calcolo delle metriche Accuracay, Precision, Recall e F1-Score per le sole Categorie

6.2 Confronto tra risultati ottenuti tramite LLM con Golden Standard per Maintenance

Il grafico sottostante mostra le prestazioni dei quattro modelli per la Maintenance, misurate con Precision, Recall, F1-Score e Accuracy considerando tutte e sei le foglie della tassonomia. Il primo dato che salta all’occhio è che qui i numeri sono decisamente più bassi rispetto a quelli visti sulle categorie: le F1-Score oscillano tra 0,30 e 0,55, mentre per quanto riguarda il grafico delle categorie, i valori oscillavano intorno a 0,62–0,68.

Guardando i singoli modelli si distinguono tre comportamenti diversi. glm-5.2:cloud e kimi-k2.7-code:cloud sono i migliori del gruppo: hanno Precision, Recall e F1-Score tutte intorno a 0,55 e un’accuratezza rispettivamente di 0,56 e 0,58, che sono i valori più alti del grafico. Il modello gemma4:31b-cloud invece, presenta un’accuratezza di 0,57, ma un F1-Score di soli 0,45. Questo significa che gemma indovina le classi frequenti (additive e perfective, che da sole fanno la maggior parte del dataset e di conseguenza, determinano un aumento dell’accuratezza) ma se la cava molto peggio sulle classi rare, che nel macro-averaging pesano quanto le altre, provocando un calo dell’F1-Score.

Particolare attenzione va al modello qwen3.5:cloud: Presenta un F1-Score pari a 0,30, un crollo netto rispetto agli altri. La motivazione è la stessa di quello che si vedeva già nel grafico precedente delle categorie: il modello non riesce a predire come adaptive i due casi in questione, e questo significa che due delle sei classi vanno a zero di Recall, diminuendo di

conseguenza la media macro. Nonostante ciò, l'accuratezza resta a 0,53.

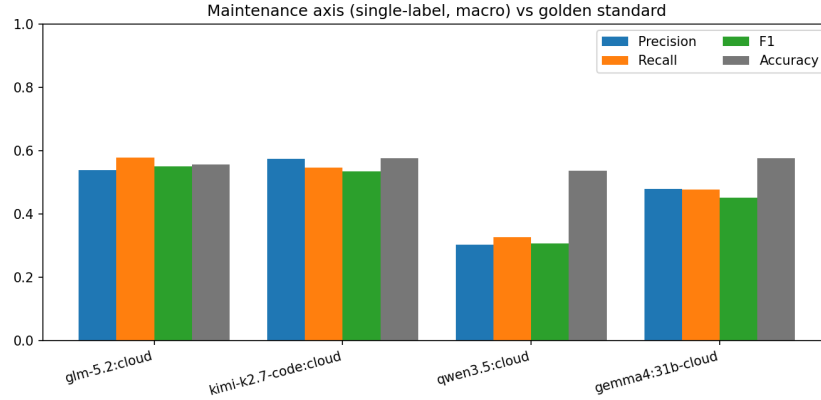


Figure 27: Confronto tra Golden Standard e Multi-LLM e calcolo delle metriche Accuraty, Precision, Recall e F1-Score per la Maintenance

Il grafico seguente si presenta come una griglia 2×2 e, per ogni pannello dedicato a ciascun modello, riporta Precision, Recall ed F1 per le sei classi di manutenzione, con il supporto nel gold annotato sotto l'etichetta di ogni classe. Le classi maggioritarie sono **perfective** ($n = 73$) e **additive** ($n = 51$), mentre **preventive** ($n = 3$), **adaptive (correction)** ($n = 6$) e **adaptive (enhancement)** ($n = 3$) sono presenti in pochi casi.

Le classi **additive** e **perfective** mostrano un F1-Score stabile tra 0,55 e 0,65 su tutti e quattro i modelli, con Recall alta per **additive** (0,70–0,79) e Precision alta per **perfective** (0,67–0,78). La classe **corrective**, pur avendo un supporto discreto ($n = 13$), si conferma la più problematica in tutti i modelli, con F1 nell'intervallo 0,26–0,41.

Il modello qwen3.5:cloud esibisce un comportamento anomalo : per la maintenance adaptive non riesce ad effettuare alcuna predizione. Di conseguenza l'analisi totalitaria viene compromessa, poichè il modello ricade sistematicamente su altre etichette. Il modello gemma4:31b-cloud, all'opposto, è l'unico modello a coprire tutti e sei i casi di Maintenance, con F1-Score non nullo, ma con Precision basse su corrective, preventive e adaptive (Correction) (fra 0,29 e 0,50).

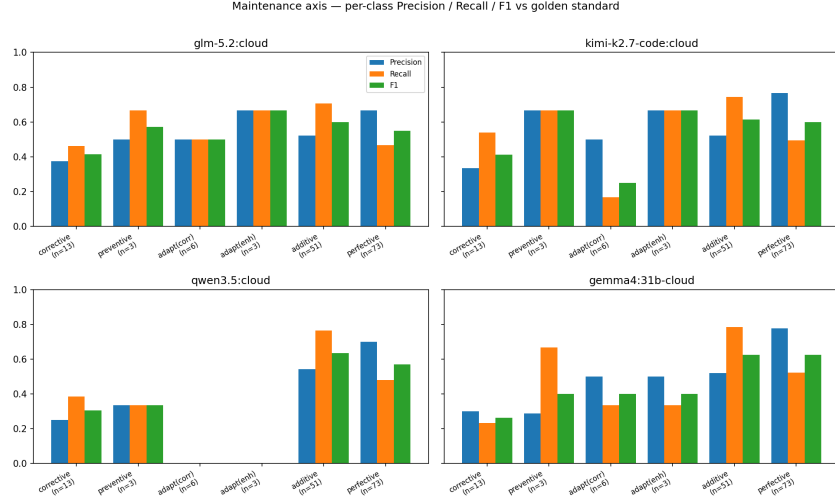


Figure 28: Grafico sulla divergenza inter-modello con errore medio dei modelli rispetto al Golden Standard

6.3 Analisi delle Ambiguità vs Errore

Il grafico seguente rappresenta la dispersione costruito sulle 150 diff del Golden Standard, che mette in relazione due grandezze calcolate per ciascuna diff. Sull'asse delle ascisse è riportata la divergenza inter-modello sulle categorie, ovvero quanto i tre modelli si trovano in disaccordo tra loro nell'assegnare le sedici etichette tematiche: viene calcolata come distanza di Jaccard media a coppie tra i tre insiemi di categorie predetti (0 se i tre modelli hanno predetto insiemi identici, 1 se sono completamente disgiunti). Sull'asse delle ordinate è invece riportato l'errore medio dei modelli rispetto al Golden Standard, calcolato come distanza di Jaccard media tra ciascun insieme predetto e l'insieme di etichette annotate manualmente. Il coefficiente di correlazione di Pearson, indicato nel titolo del grafico, vale $r = 0,37$: si tratta di una correlazione positiva di intensità moderata, non trascurabile dal punto di vista statistico.

Le diff sulle quali i tre modelli concordano fra loro si trovano nella parte sinistra del grafico e tendono ad addensarsi nella metà inferiore dell'ordinata, dove l'errore medio contro il Golden Standard è contenuto. Spostandosi verso destra, all'aumentare della divergenza (in particolare per valori superiori a 0,5), la nube dei punti si sposta verso l'alto e l'errore contro il gold cresce, con molti casi che superano il valore di 0,6. In altri termini, quando i tre modelli non riescono a mettersi d'accordo tra loro, tendono anche a sbagliare rispetto all'annotazione umana; viceversa, quando concordano, la loro predizione risulta generalmente corretta.

La dispersione dei punti resta comunque ampia e la correlazione non è perfetta. Sono infatti presenti alcuni casi limite con divergenza nulla ma errore massimo, che corrispondono a istruzioni sulle quali tutti e tre i modelli concordano su una classificazione sbagliata. Questo residuo mostra che il disaccordo inter-modello è un buon indicatore dell'ambiguità semantica del testo analizzato, ma non ne è un sostituto perfetto: esistono forme di ambi-

guità che i tre modelli condividono e interpretano allo stesso modo e che quindi non sarebbe in grado di rilevare nessun voto a maggioranza.

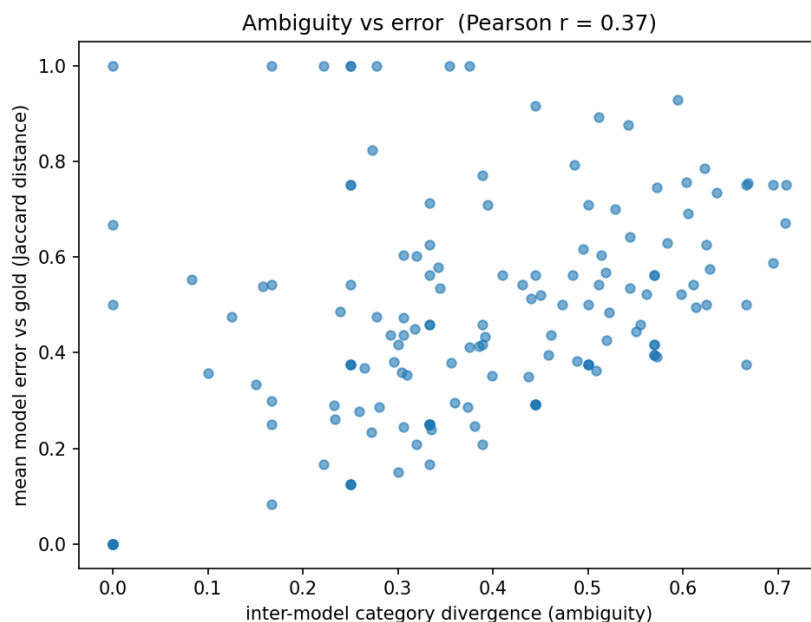


Figure 29: Confronto tra Golden Standard e Multi-LLM e calcolo delle metriche Precision, Recall e F1-Score per singole classi di Maintenance

7 Conclusioni

Il progetto ha affrontato il problema di analizzare in modo sistematico l'evoluzione degli Agent Context File, superando l'approccio empirico prevalente in letteratura e proponendo una pipeline strutturata che unisce estrazione automatica, classificazione tramite LLM e validazione multi-modello. Il percorso si è articolato in tre fasi complementari: l'estrazione delle diff dallo storico dei commit tramite le API di GitHub, la classificazione delle modifiche lungo i due assi delle categorie tematiche e dei maintenance type ISO/IEC/IEEE 14764 tramite un LLM primario guidato da prompt strutturato, e infine la valutazione della robustezza delle predizioni tramite Cross-Model Validation su tre modelli di studio e confronto con un Golden Standard costruito manualmente.

I risultati ottenuti mostrano che l'approccio adottato è complessivamente affidabile ma con differenze marcate tra i due assi analizzati. Sull'asse tematico, i quattro modelli si comportano in maniera solida e allineata: l'accordo inter-modello si attesta su valori sostanziali (Kappa di Fleiss tra 0,70 e 0,74) e il confronto con il Golden Standard restituisce un'accuratezza costante attorno all'85% per tutti i modelli, con F1-Score tra 0,62 e 0,68. Questi valori confermano che l'estrazione delle diff è un compito ben definito, per il quale il vocabolario e le direttive presenti negli ACF risultano sufficientemente espliciti da consentire una classificazione oggettiva.

Il quadro cambia sull'asse della manutenzione, dove i modelli devono inferire **non cosa è stato modificato** ma **perché lo si è fatto**. Qui le F1-Score scendono nella fascia 0,30-0,55, e le classi rare come preventive e le due varianti adaptive mettono in difficoltà quasi tutti i modelli.

Fra i tre modelli di studio, glm-5.2:cloud e kimi-k2.7-code:cloud si sono dimostrati i più prestanti su entrambi gli assi, con un profilo bilanciato tra Precision e Recall. Il modello primario gemma4:31b-cloud, pur restando affidabile grazie alla sua architettura più imponente, ha mostrato una tendenza permissiva che gli garantisce Recall elevate ma penalizza la Precision. qwen3.5:cloud, al contrario, si è rivelato il modello più debole del gruppo: sull'asse della maintenance non è mai in grado di predire come maintenance adaptive, con la conseguenza che due classi su sei registrano Recall pari a zero e trascinano al ribasso la media macro fino a un F1-Score di 0,30, un valore nettamente distante da quello degli altri modelli.

8 Sviluppi futuri

Sulla base delle osservazioni raccolte, il primo intervento naturale riguarda proprio la sostituzione di qwen3.5:cloud all'interno del pool di modelli di studio. Come emerso dall'analisi di confronto con il Golden Standard, questo modello è risultato sistematicamente il peggiore dei quattro, sia sull'asse tematico che, in misura molto più marcata, sull'asse della manutenzione, ovvero le predizioni errate per adaptive che ha compromesso l'intero confronto. La sua sostituzione con un LLM di fascia comparabile ma più affidabile — ad esempio un modello della famiglia DeepSeek, Mistral Large o una variante più recente della stessa famiglia Qwen — permetterebbe di ricalcolare le metriche di accordo cross-model su una base più solida, riducendo il rumore che qwen introduce nelle medie inter-pool e nelle correlazioni fra divergenza ed errore.

Un secondo filone di sviluppo riguarda l'ampliamento della finestra temporale di estrazione delle diff. Come già osservato nel paragrafo §4.6.9, la scarsità della classe adaptive (4,2% del corpus) è in parte imputabile al numero limitato dei commit fin'ora osservati, che coglie solo parzialmente gli adeguamenti a toolchain e ambienti in evoluzione. Estendere l'analisi a una finestra temporale più ampia permetterebbe di ottenere una stima più aderente alla reale frequenza dei fenomeni adaptive, con un impatto diretto sia sulla distribuzione osservata dei maintenance type sia sul bilanciamento delle classi rare nel Golden Standard.

Un terzo sviluppo potrebbe consistere nell'ampliare la dimensione del Golden Standard oltre le attuali 150 diff. Il campione attuale rappresenta un buon compromesso tra sforzo di annotazione e significatività statistica, ma le classi con supporto minimo (come preventive con sole 3 occorrenze o adaptive (enhancement) sempre con 3) restano fragili: un singolo errore su queste classi produce oscillazioni metriche notevoli e rende poco robuste le conclusioni sulle prestazioni per singola classe. Un'annotazione manuale su 300-400 diff, offrirebbe una base di riferimento più solida e permetterebbe un miglioramento nella distinzione dall'errore del modello all'analisi manuale da parte degli sviluppatori.

Infine, un possibile sviluppo potrebbe consistere nel passare da un’analisi descrittiva dell’evoluzione degli ACF a una predittiva: date le prime versioni di un file di contesto e la traiettoria delle modifiche precedenti, un modello opportunamente addestrato potrebbe suggerire quali direttive sono candidate a essere riformulate, integrate o corrette, trasformando l’analisi in uno strumento operativo di supporto agli sviluppatori.

References

- [1] Worawalan Chatlatanagulchai et al. “Agent READMEs: An Empirical Study of Context Files for Agentic Coding”. In: *arXiv preprint arXiv:2511.12884* (2025). URL: <https://arxiv.org/abs/2511.12884>.
- [2] “Software Engineering — Software Life Cycle Processes—Maintenance”. In: *ITeh Standards* (2022). URL: <https://cdn.standards.iteh.ai/samples/80710/9a26fc3a1a464fa286fb85d6c1eed31/ISO-IEC-IEEE-14764-2022.pdf>.
- [3] Yong Wu et al. “ContextBudget: Budget-Aware Context Management for Long-Horizon Search Agents”. In: *arXiv preprint arXiv:2604.01664* (2026). URL: <https://arxiv.org/abs/2604.01664>.

Link alla Repository

E’ possibile visionare il progetto al seguente link: [ContextWare](#)